多层感知机与卷积神经网络

授课教师：Hao Wang

AI Thrust，信息枢纽



上周回顾



线性分类器



一个更典型的框架

K 最近邻

```
def train(images, labels):
  # Machine learning!
  return model
```



记忆所有数据和标签

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



预测最相似训练图像的标签



一个更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```



定义一个模型 $f_W(x)$ 定义损失函数 L $f_W(x)$, y求解 W^*，使得 L 最小

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



基于 $f_{W^*}(x)$预测标签



正则化

L2 正则化： $e\ W\ (\ \ W\ \#\ (\ \in_k W_k^2 L_1$ 正则化： $e\ W\ (\ \ W\ \%\ (\ \in_k W_k$ | |

1. “最大间隔”原则

在 SVM 中， 目标是找到一个能够分离各类别的决策边界 d

不使用正则化： 模型可能会过度努力地正确分类训练数据中的每一个噪声点，从而导致非常复杂的边界。

使用正则化： 最小化 $W\ _2$ 在数学上等价于 **最大化间隔**。 更大的间隔通常会在新的、未见过的数据上带来更好的性能，因为它更具鲁棒性。

2. 控制“权重”

为什么需要小权重? 如果某个权重 W_i 非常大，那么即使特定输入特征 x_i 发生极微小的变化，也会导致输出出现巨大波动。通过迫使权重保持较小，正则化可确保模型不会过度 a

依赖 训练集中的任何单一特征或特定噪声。这会带来一个**更平滑**的模型。



损失函数

比较两个概率分布：交叉熵

$$L\left(=\frac{3}{W}\right)_i \; x_i \, \text{qpl}\,(y_i)\left(=\frac{3}{W}\right)_i \; \text{qpl}\left(\frac{h^{s_{y_i}}}{\in_j h^{s_j}}\right)$$



概率必须为 □□ =

概率之和必须为 1

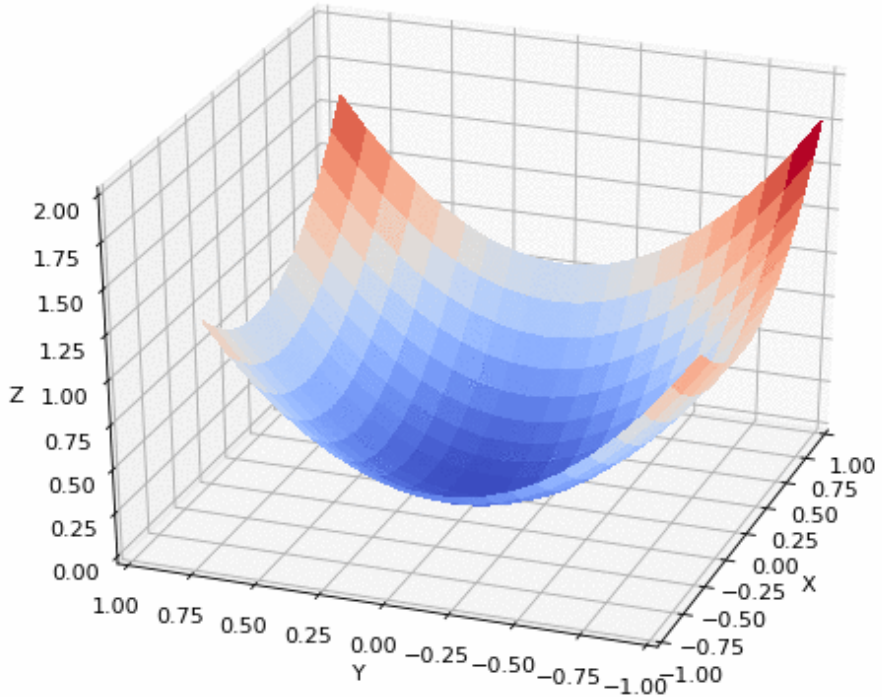
cat	**3.2**	exp	**24.5**	归一化	**0.13**	**1**
car	5.1		164.0		0.87	**0**
frog	-1.7		0.18		0.00	0

比较

x ? $s(=_i, f\ )$?¯

$$\frac{?^{\neg k}}{\sum_{+}?^{\neg j}}$$

梯度下降



思路：沿着使损失下降最陡的方向前进。

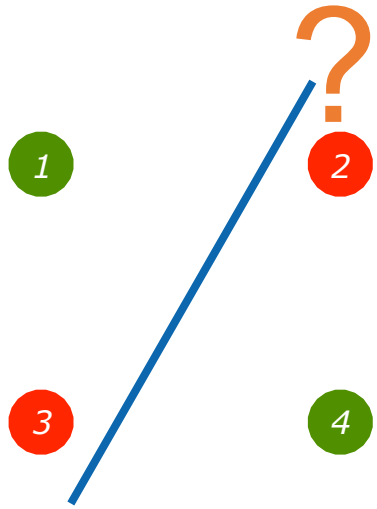
$$W\ L\ W = \boldsymbol{x}\frac{jL}{jW}$$

步长　　梯度

本周：非线性模型

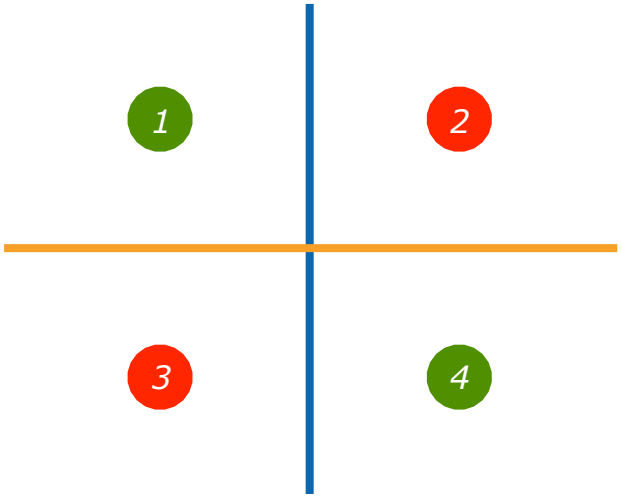


学习 XOR



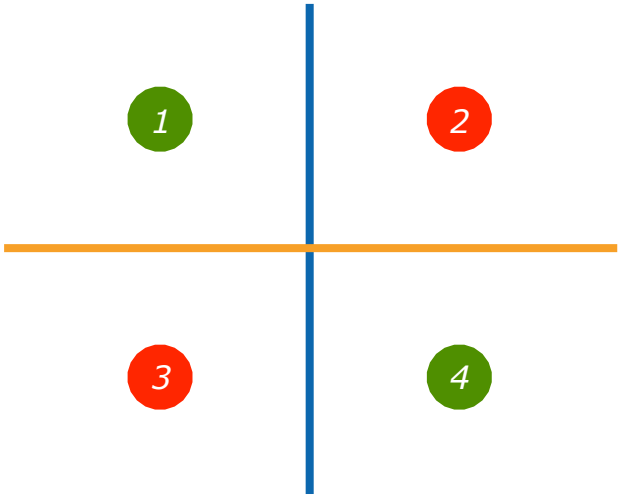
线性分类器无法处理 XOR 问题

学习XOR





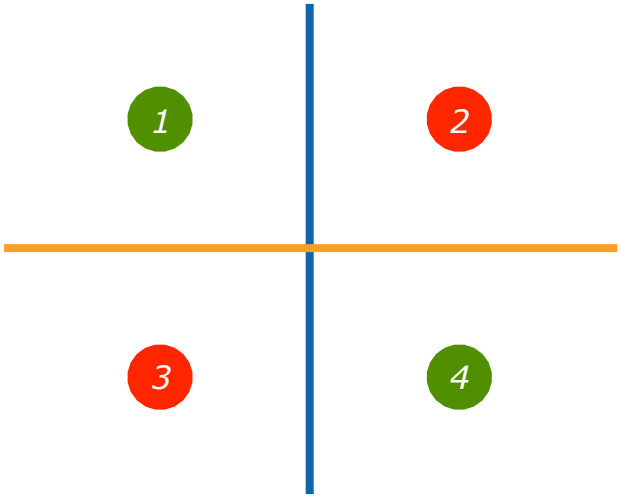
学习异或



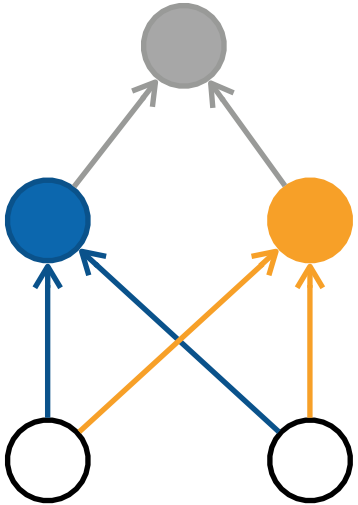
	2	3	4	6
	+	-	+	-
	+	+	-	-
乘积	+	-	-	+

通过结合这两个线性特征，我们成功地将绿色点 (□) 与红色点 (-) 分离开来!

学习XOR



	2	3	4	6
	+	-	+	-
	+	+	-	-
乘积	+	-	-	+

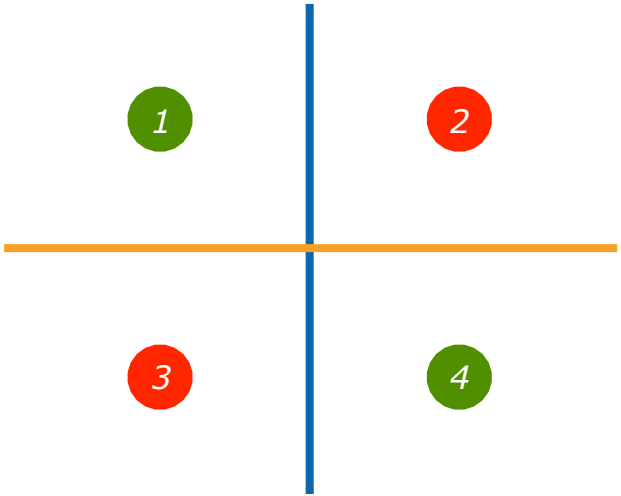


输出层： 该节点接收隐藏层的输出并将它们组合起来以进行最终分类（表格的最后几行）。

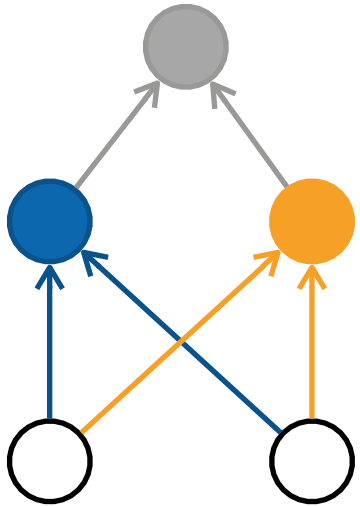
隐藏层：这些节点表示图中绘制的两条线。它们接收原始坐标，并应用线性变换（表的前两行）。

输入层：这些表示数据点的原始 x 和 y 坐标。

学习异或



	2	3	4	6
	+	-	+	-
	+	+	-	-
乘积	+	-	-	+

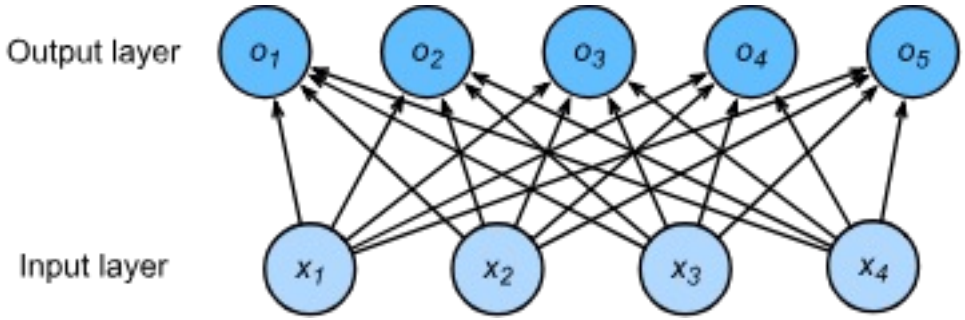


问题： 这是一个线性模型吗？为什么？

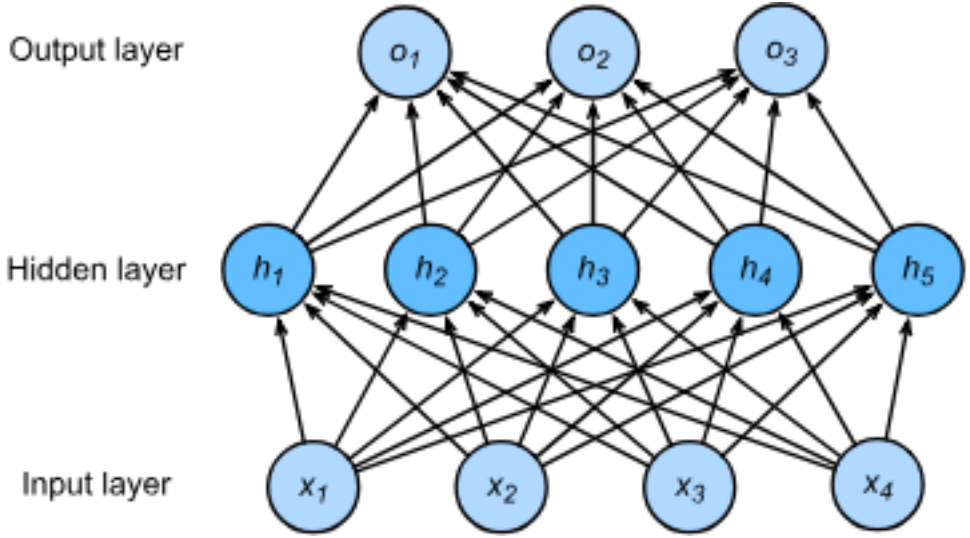
答案：不！

- **阈值**（激活函数，阶跃函数）
- **乘积运算**（将变量相乘，如 $x_1 \times x_2$）或应用逻辑规则是一种根本上的非线性运算。）

单隐藏层



单隐藏层



超参数——隐藏层大小 m

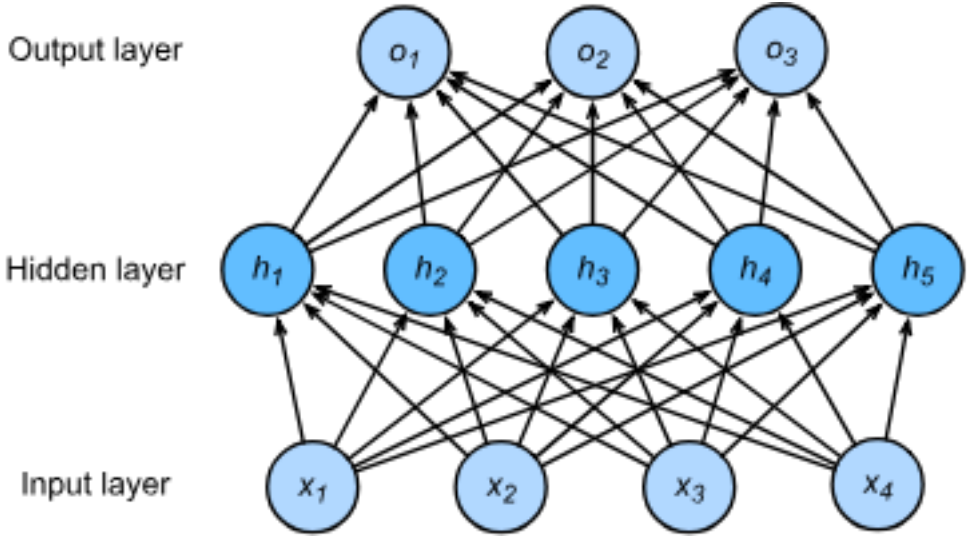
单隐藏层

- 输入 $\mathbf{x} \in \mathbb{R}^n$
- 隐藏 $\mathbf{W}_1 \in \mathbb{R}^{m \times n} \mathbf{b}_1 \in \mathbb{R}^m$
- 输出 $\mathbf{W}_2 \in \mathbb{R}^m$, $\mathbf{b}_2 \in \mathbb{R}$

$$\mathbf{h} = \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1)$$

$$o = \mathbf{w}^T \mathbf{h} + b_2$$

σ 是一种逐元素激活函数



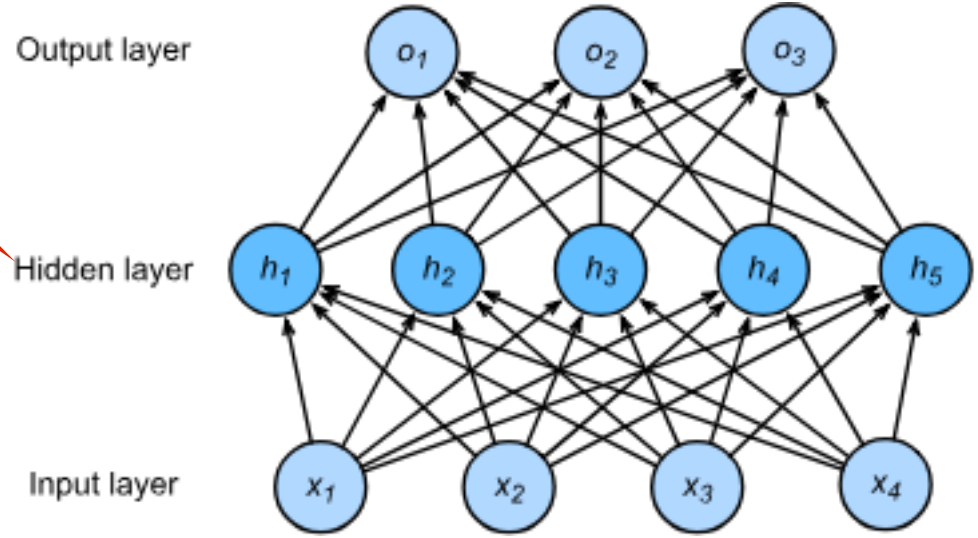
单隐藏层

为什么我们需要非线性激活?

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}^T\mathbf{h} + b_2$$

σ 是一个逐元素的激活函数



单隐藏层

线性变换的线性变换是 仍然是线性变换!

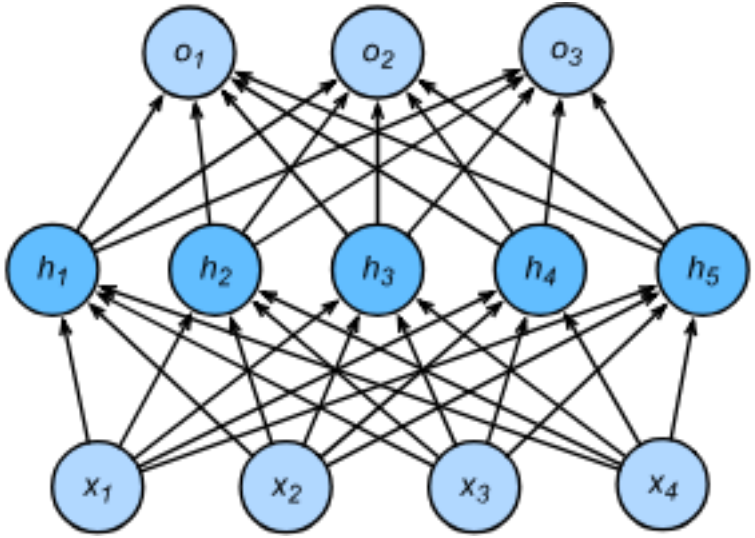
为什么我们需要非线性激活?

$$\mathbf{h} = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$$

$$\mathbf{o} = \mathbf{w}^T\mathbf{h} + b_2$$

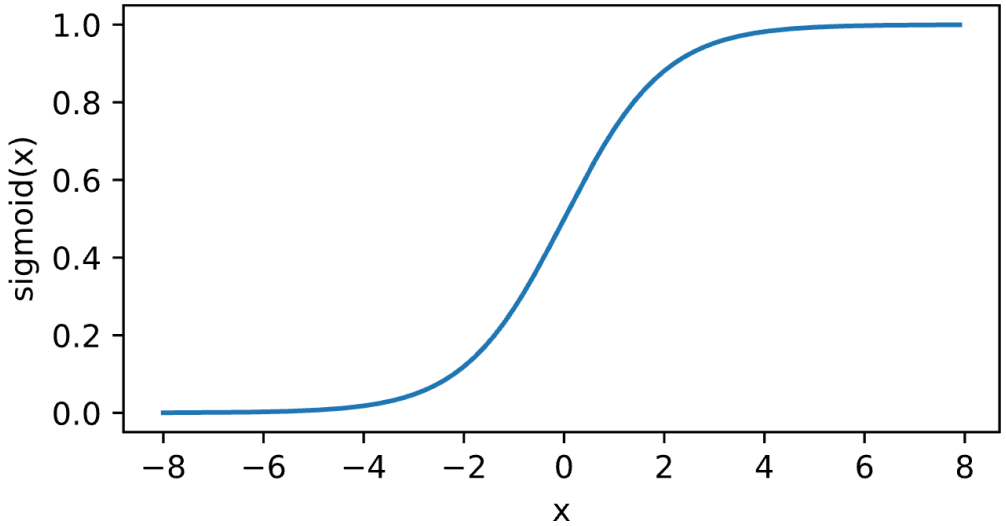
因此 $o = \mathbf{w}^T\mathbf{W}_1\mathbf{x} + b'$

线性 ...



Sigmoid 激活

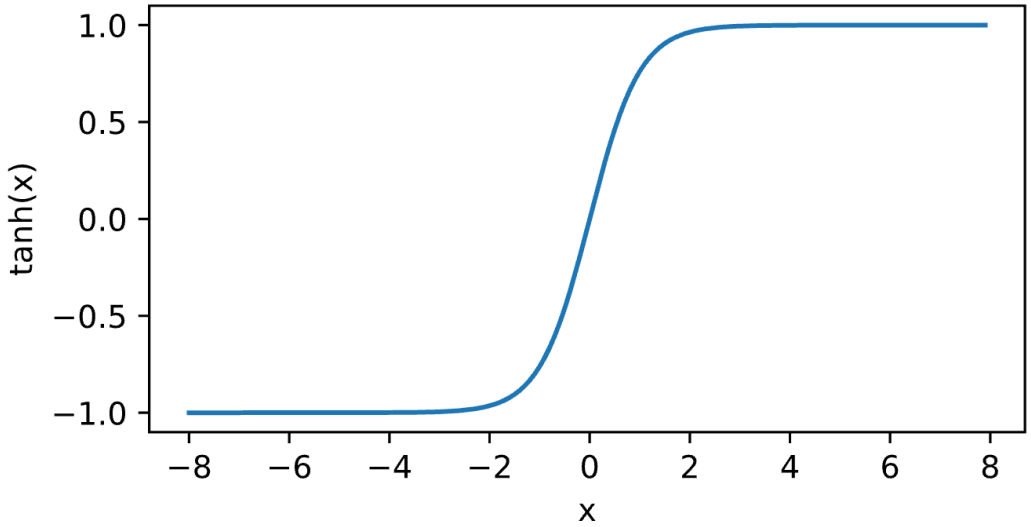
将输入映射到 (0, 1)，一种平滑版本：$\sigma(x) = \frac{1}{1+\exp(-x)}$，如果 x 是 $n \times 3$ 矩阵，则 $\sigma(x)$ 是 $n \times 3$ 矩阵



Tanh 激活函数

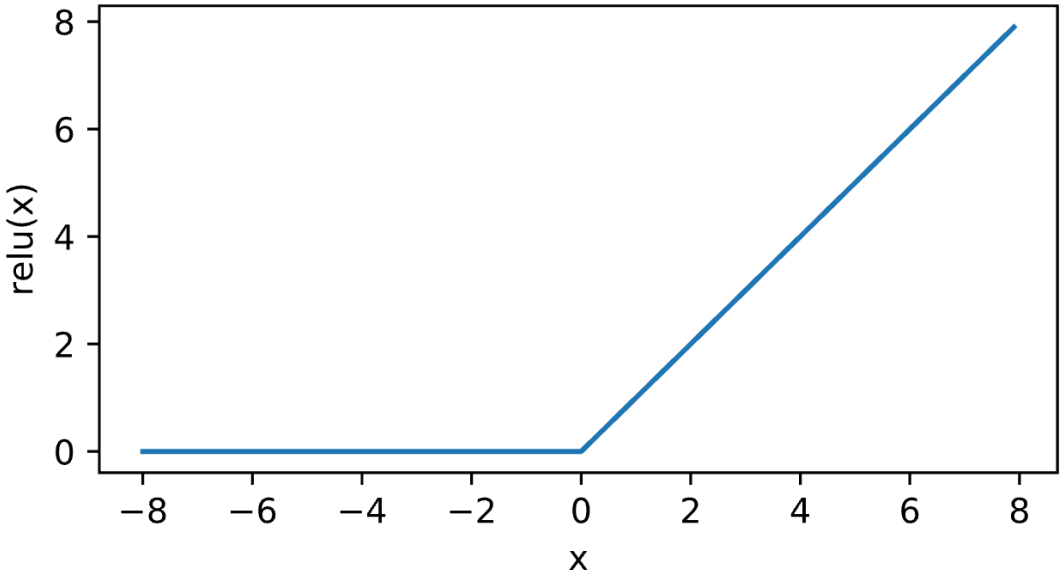
将输入映射到 (-1, 1)

$$\square anh \gg x \square\square \quad \frac{3 = g0x(=70)}{3 \times g0x(=70)}$$



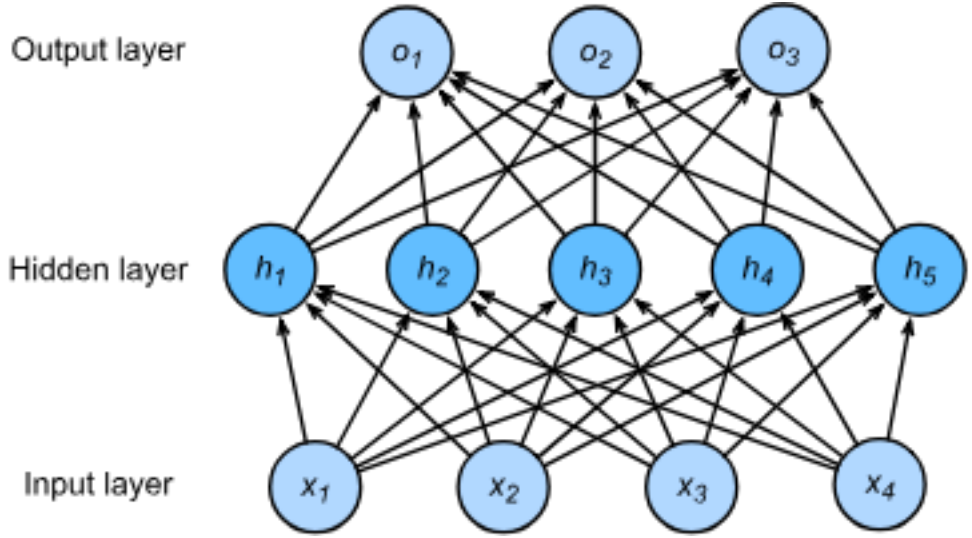
ReLU 激活

ReLU：修正线性单元：ReLU(x) ≜ max(x,0)



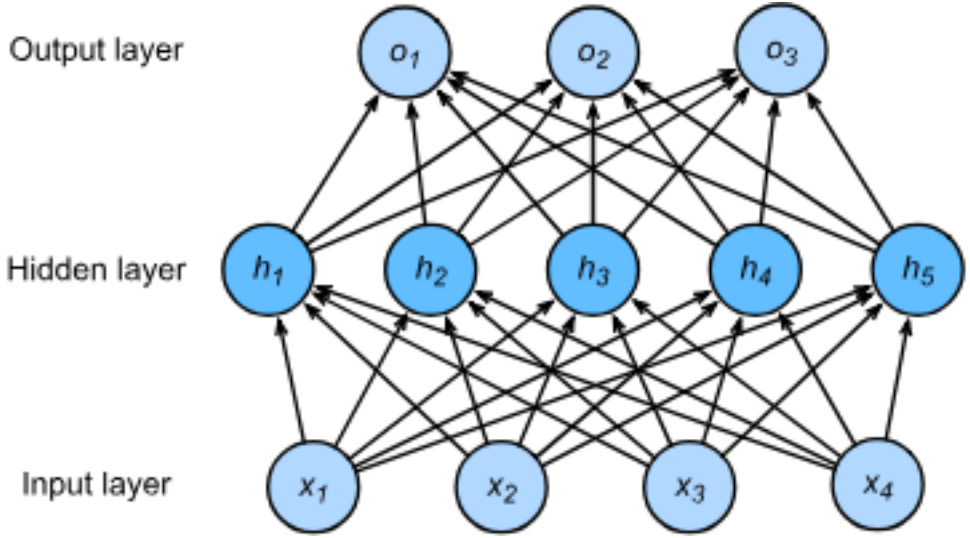


多类分类


Output layer
Hidden layer
Input layer
o_1
o_2
o_3
h_1
h_2
h_3
h_4
h_5
x_1
x_2
x_3
x_4

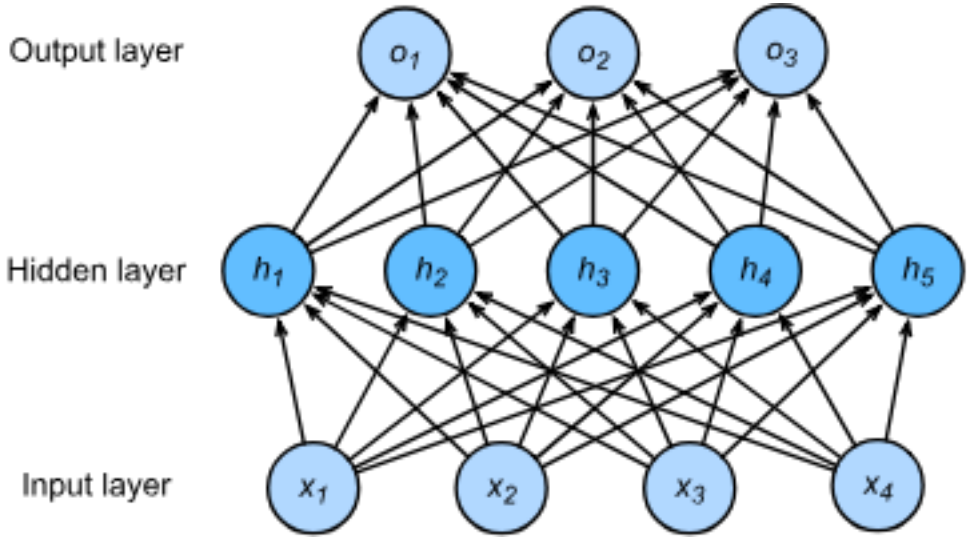
多类分类

$y_1 1 y_2 1 \ldots, y_k \ ? \ \text{softmax}(o_1 1 o_2 1 \ldots, o_k)$



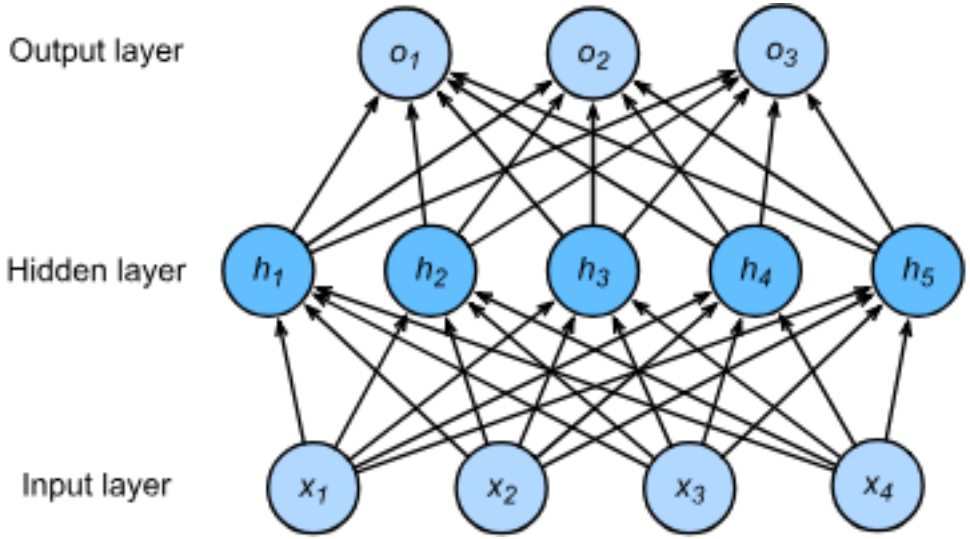
多类分类

- 输入 $\mathbf{x} \, e \, \mathrm{R}n$
- 隐藏 $\mathbf{W}_1 \in \mathrm{R}^{m \times n}$ 和 $\mathbf{b}_1 \in \mathrm{R}^m$
- 输出 $\mathbf{W}_2 \in \mathrm{R}^{m \times d}$ 和 $\mathbf{b}_2 \in \mathrm{R}^d$



多类分类

- 输入 $\mathbf{x} \in \mathbb{R}^n$
- 隐藏层 $\mathbf{W}_1 \in \mathbb{R}^{m \times n}$ 和 $\mathbf{b}_1 \in \mathbb{R}^m$
- 输出层 $\mathbf{W}_2 \in \mathbb{R}^{m \times d}$ 和 $\mathbf{b}_2 \in \mathbb{R}^d$



$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\mathbf{o} = \mathbf{w}^T\mathbf{h} + \mathbf{b}$$

$$\mathbf{y} = \text{softmax}(\mathbf{o})$$

多个隐藏层

$$\mathbf{h}_1 = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

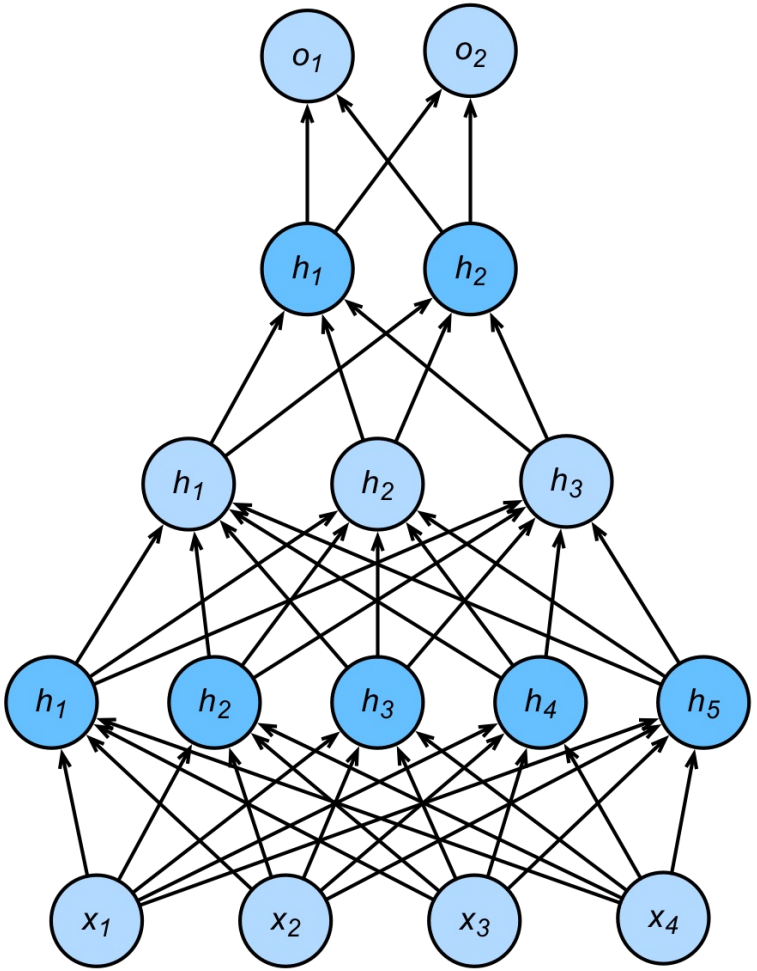
$$\mathbf{h}_2 = \sigma(\mathbf{W}_2\mathbf{h}_1 + \mathbf{b}_2)$$

$$\mathbf{h}_3 = \sigma(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3)$$

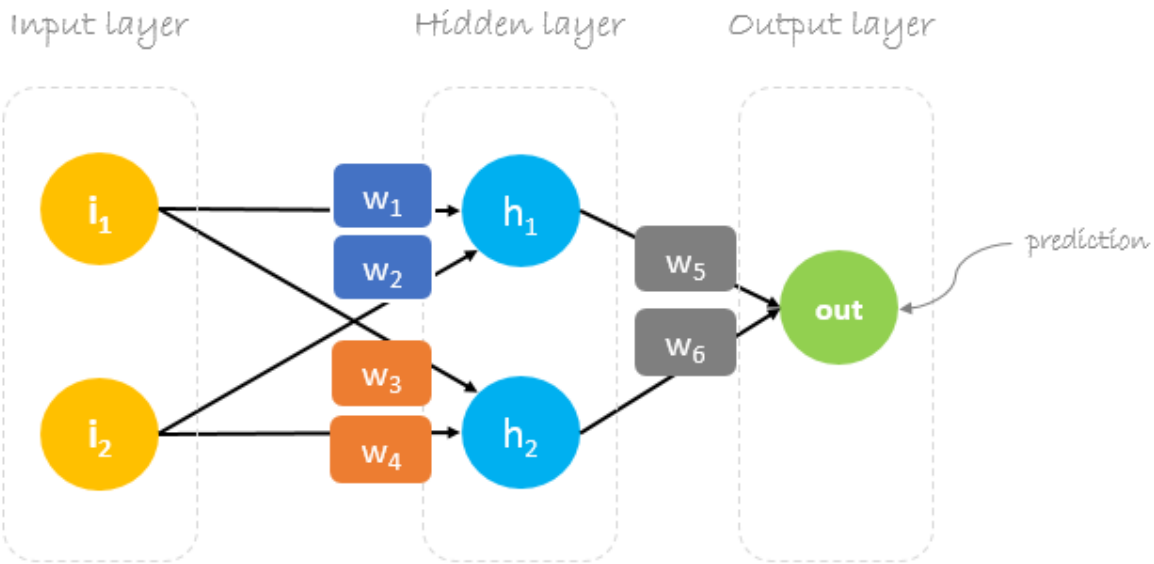
$$\mathbf{o} = \mathbf{W}_4\mathbf{h}_3 + \mathbf{b}_4$$

超参数

- 隐藏层数量
- 每一层的隐藏层大小

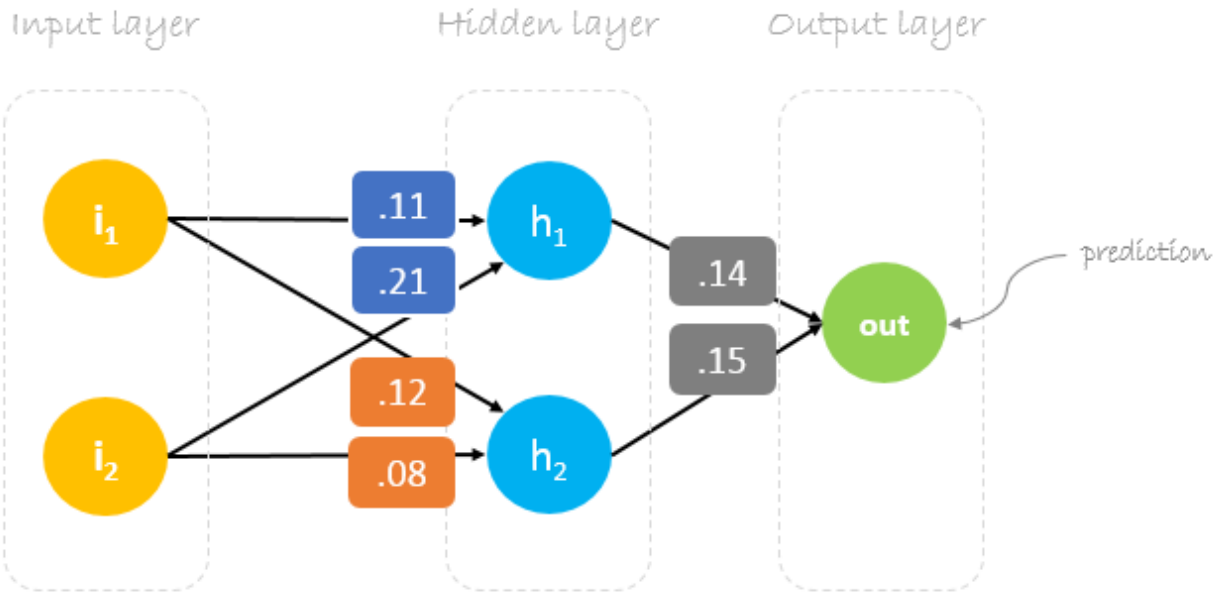


训练 MLP：反向传播



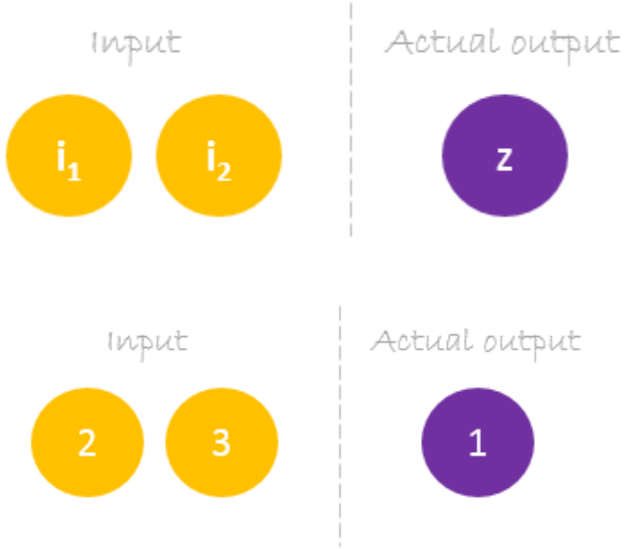
https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



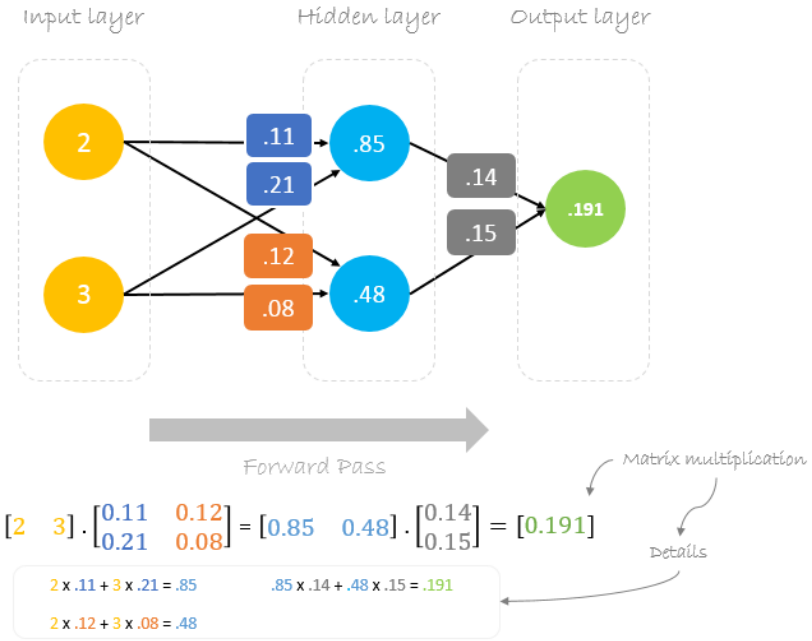
https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



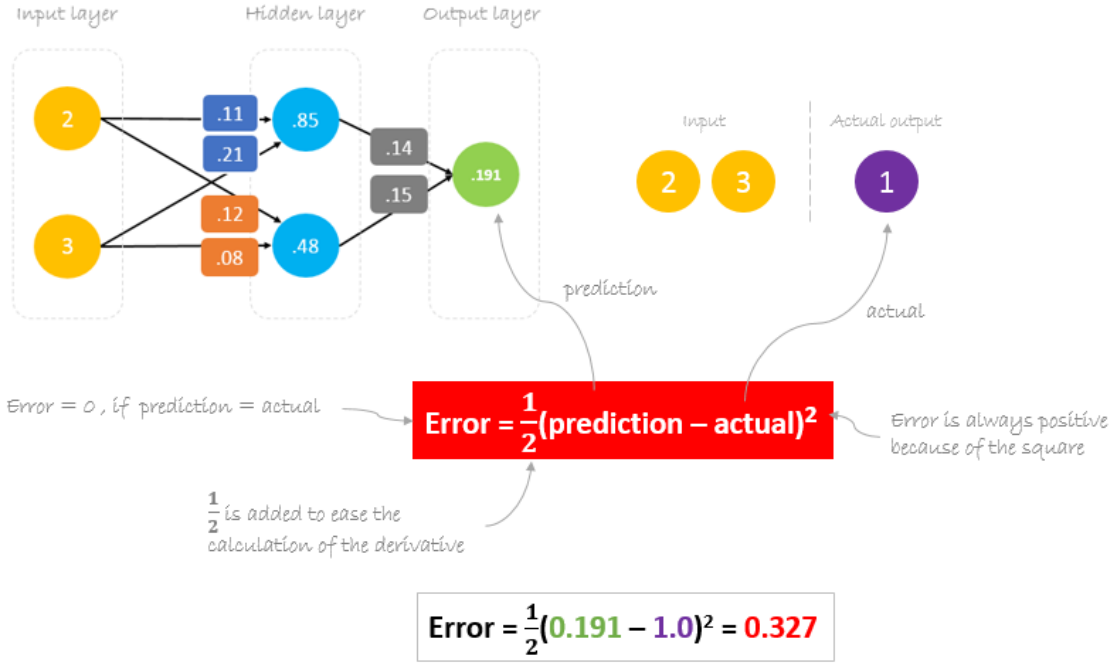
https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$$\text{prediction} = \text{out}$$

$$\text{prediction} = (h_1)\, w_5 + (h_2)\, w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$

$$h_2 = i_1 w_3 + i_2 w_4$$

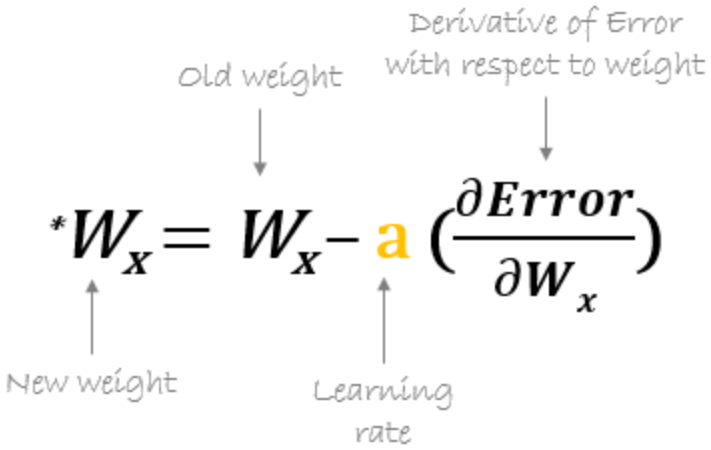
$$\text{prediction} = (i_1 w_1 + i_2 w_2)\, w_5 + (i_1 w_3 + i_2 w_4)\, w_6$$

to change ***prediction*** value,
we need to change ***weights***



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$$^{*}W_6 = W_6 - \mathrm{a}\left(\frac{\partial Error}{\partial W_6}\right)$$



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$$\frac{\partial Error}{\partial W_6} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial W_6}$$

chain rule

Error = $\frac{1}{2}$(prediction – actual)^2

prediction = $(i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$

$$\frac{\partial Error}{\partial W_6} = \frac{\frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6}{\partial W_6}$$

$$\frac{\partial Error}{\partial W_6} = 2 * \frac{1}{2}(\mathbf{predictoin - actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (i_1 w_3 + i_2 w_4)$$

$h_2 = i_1 w_3 + i_2 w_4$

$$\frac{\partial Error}{\partial W_6} = (\mathbf{predictoin - actula}) * (h_2)$$

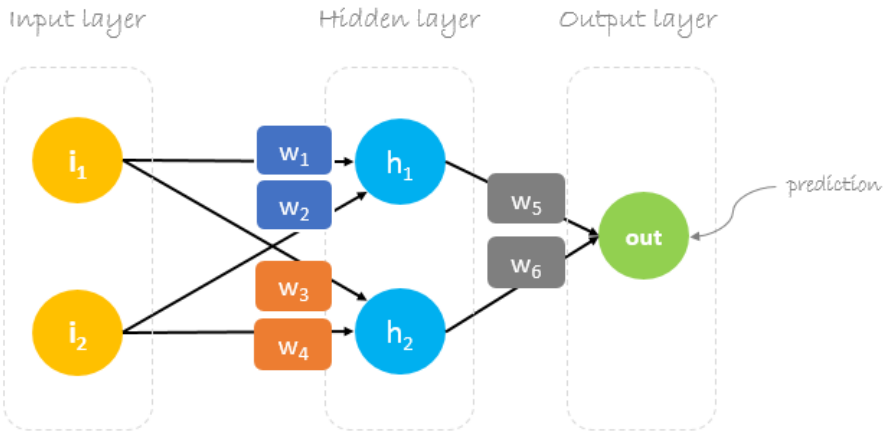
Δ = prediction – actual

delta

$$\frac{\partial Error}{\partial W_6} = \Delta h_2$$

https://hmkcode.com/ai/backpropagation-step-by-step/

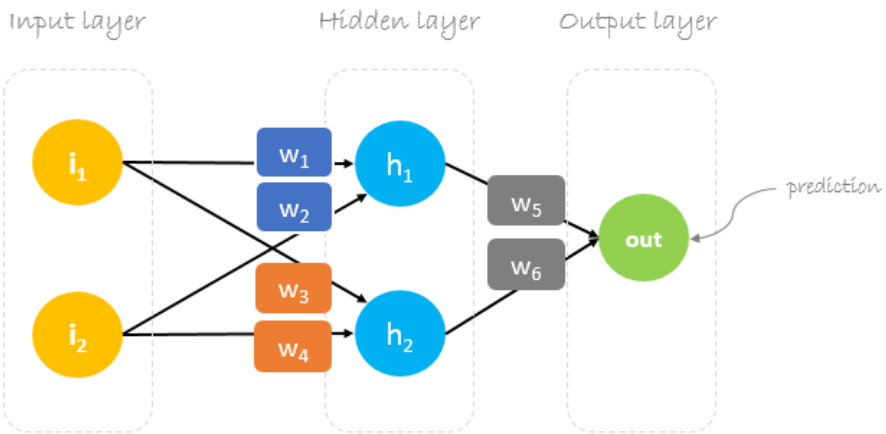
训练 MLP：反向传播



$$*W_6 = W_6 - a\,\Delta h_2$$

https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



$$^*W_5 = W_5 - a\,\Delta h_1$$

https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$$\frac{\partial Error}{\partial W_1} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial h_1} * \frac{\partial h_1}{\partial W_1}$$

chain rule

$$\frac{\partial Error}{\partial W_1} = \frac{\partial \frac{1}{2}(predictoin - actula)^2}{\partial prediciton} * \frac{\partial (h_1)\, w_5 + (h_2)\, w_6}{\partial h_1} * \frac{\partial\, i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial Error}{\partial W_1} = 2 * \frac{1}{2}(\mathbf{predictoin - actula}) \frac{\partial (predictoin - actula)}{\partial prediciton} * (w_5) * (i_1)$$

$$\frac{\partial Error}{\partial W_1} = (\mathbf{predictoin - actula}) * (w_5 i_1)$$

$$\frac{\partial Error}{\partial W_1} = \Delta\, w_5 i_1$$

$\mathbf{Error = \frac{1}{2}(prediction - actual)^2}$

$\mathbf{prediction} = (h_1)\, w_5 + (h_2)\, w_6$

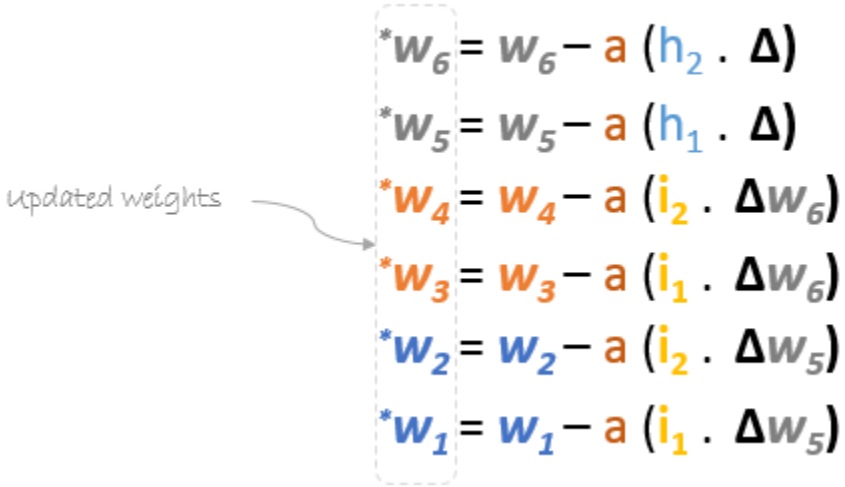
$h_1 = i_1 w_1 + i_2 w_2$

Δ = prediction – actual

delta

https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a\,\Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a\,\Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} . [w_5 \quad w_6] = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a\, i_1 \Delta w_5 & a\, i_1 \Delta w_6 \\ a\, i_2 \Delta w_5 & a\, i_2 \Delta w_6 \end{bmatrix}$$



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播

$\Delta = 0.191 - 1 = -0.809$ ← **Delta** = prediction - actual

$a = 0.05$ ← **Learning rate,** we smartly guess this number

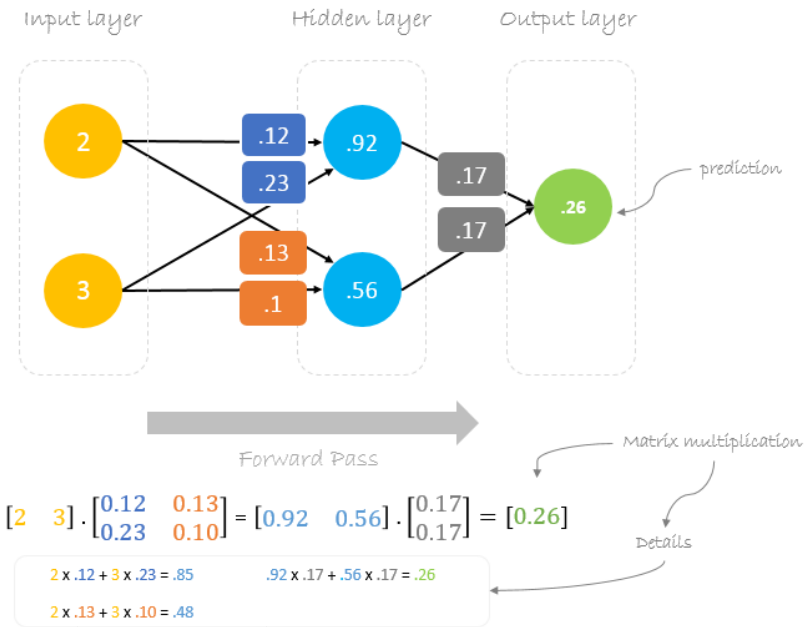
$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} . \begin{bmatrix} 0.14 & 0.15 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$



https://hmkcode.com/ai/backpropagation-step-by-step/

训练 MLP：反向传播



使用更新后的权重，预测值为 0.26（此前为 0.191），更接近真实值 1。

https://hmkcode.com/ai/backpropagation-step-by-step/

演练场

- https://playground.tensorflow.org

- 使用默认设置查看其行为
- 将 Sigmoid 改为 tanh
- 使用 sigmoid/Tanh 和 5 层，以查看学习行为
- 将其改为 ReLU。

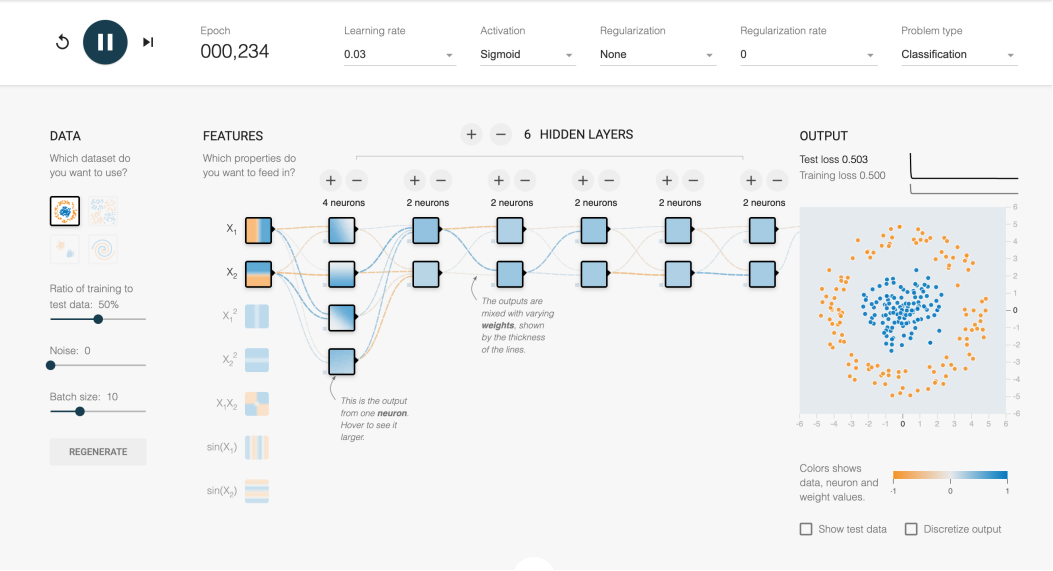


激活函数



一些观察

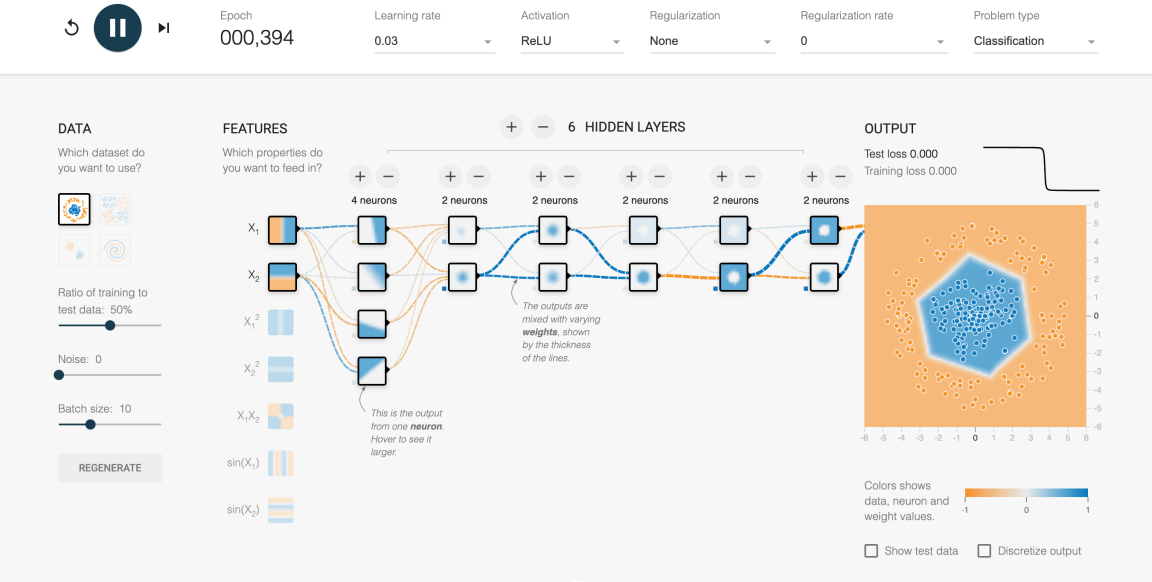
- 如果使用很多层，网络不会收敛



https://playground.tensorflow.org

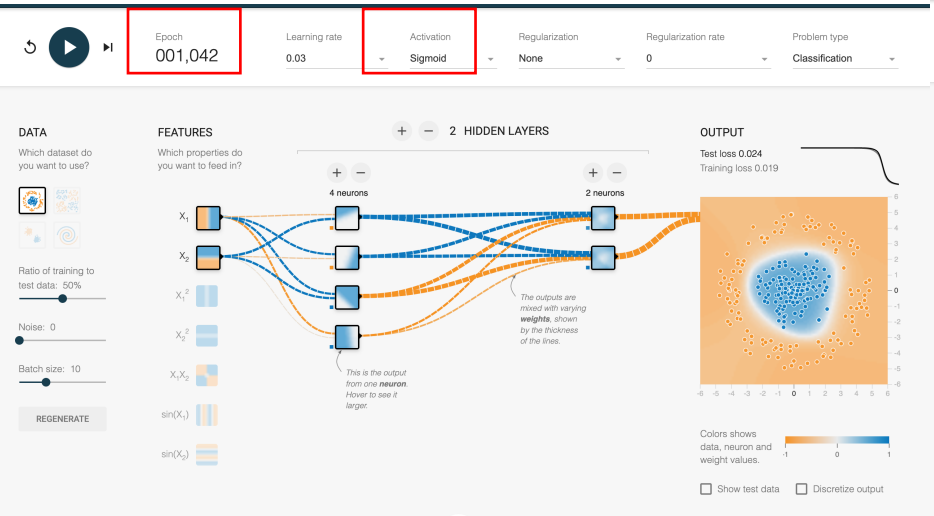
一些观察

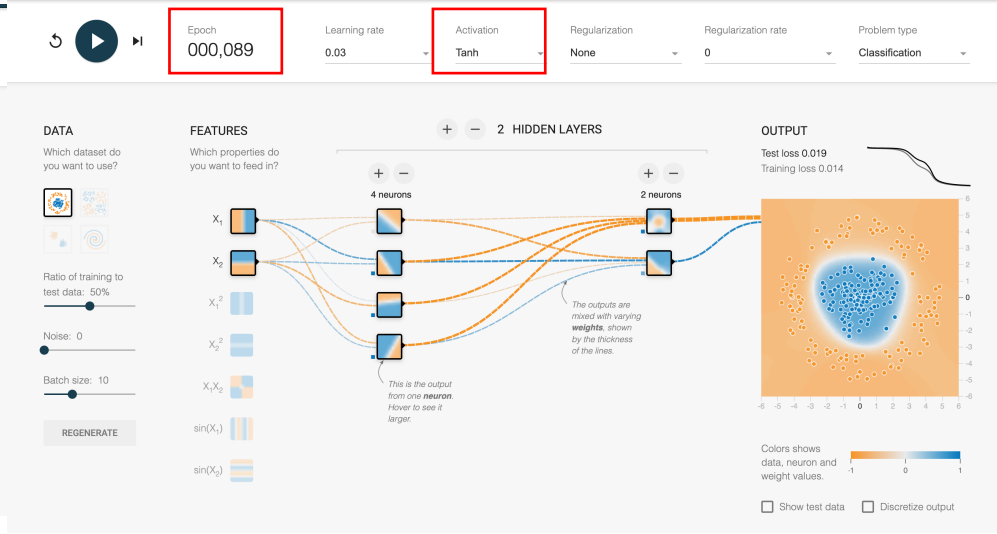
- 将 Sigmoid 改为 ReLU 可解决此问题



一些观察

- Tanh 的学习速度比 sigmoid 更快

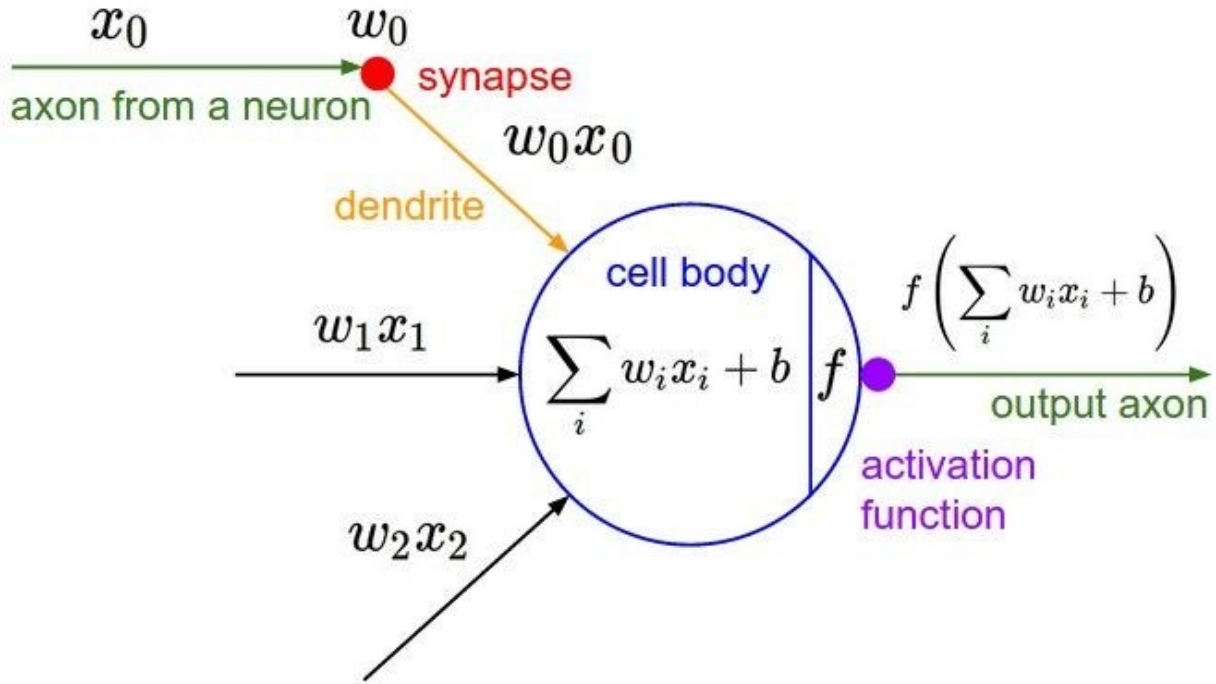




激活函数



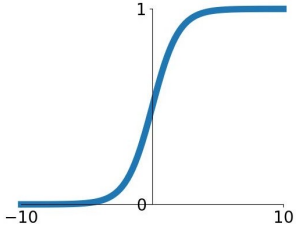
激活函数


x_0
w_0
axon from a neuron
synapse
$w_0 x_0$
dendrite
cell body
$w_1 x_1$
$\sum_i w_i x_i + b$
f
$f\left(\sum_i w_i x_i + b\right)$
output axon
activation function
$w_2 x_2$

激活函数

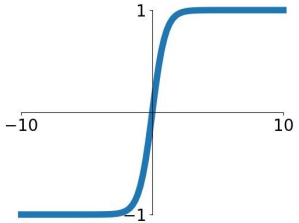
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



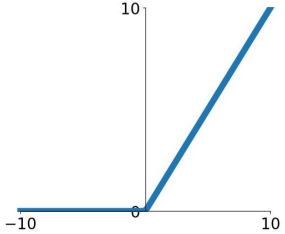
tanh

$$\tanh(x)$$



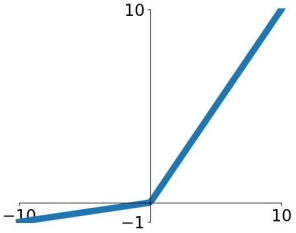
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

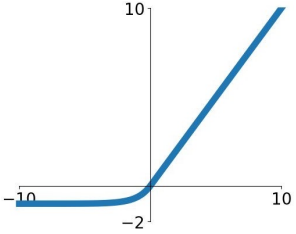


Maxout

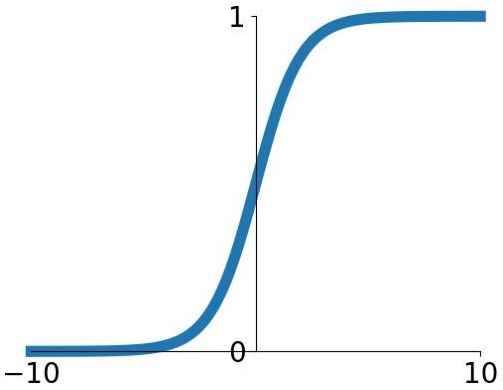
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



激活函数



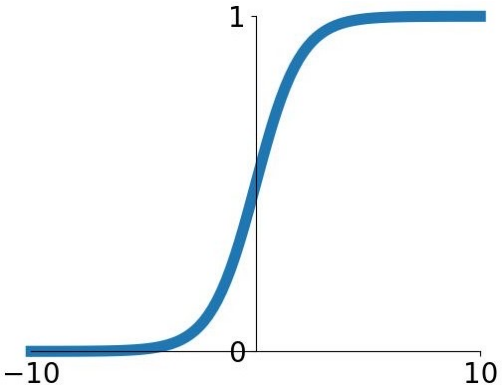
Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

- 将数值压缩到范围 0 - 历史上很受欢迎，因为它们可以很好地解释为神经元的饱和“发放率”

激活函数

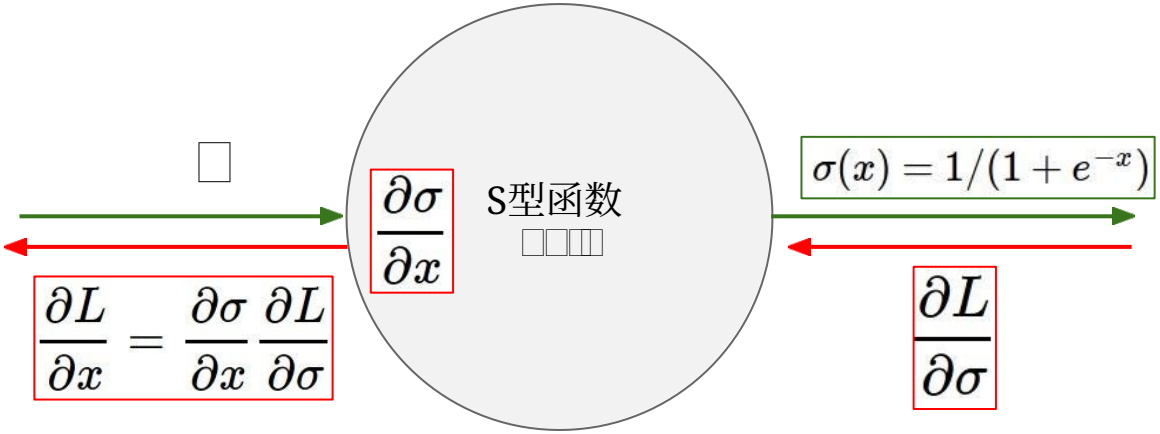
$$\sigma(x) = 1/(1+e^{-x})$$

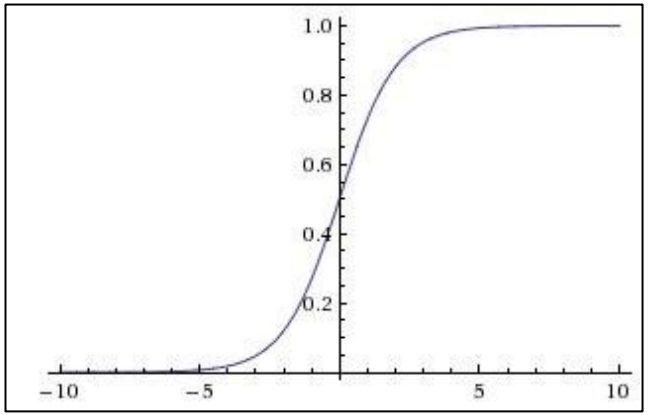


Sigmoid

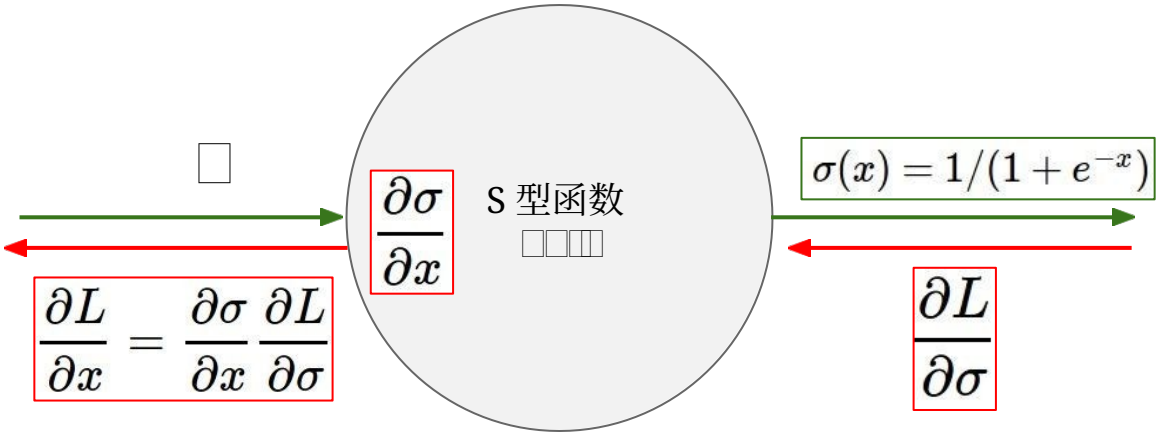
- 将数值压缩到范围 0 由于它们可以很好地解释为神经元的饱和“发放率”，因此历来很受欢迎

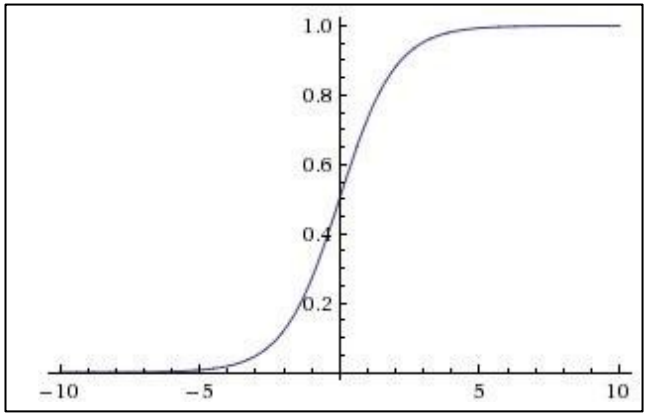
1. 饱和神经元会“杀死”梯度





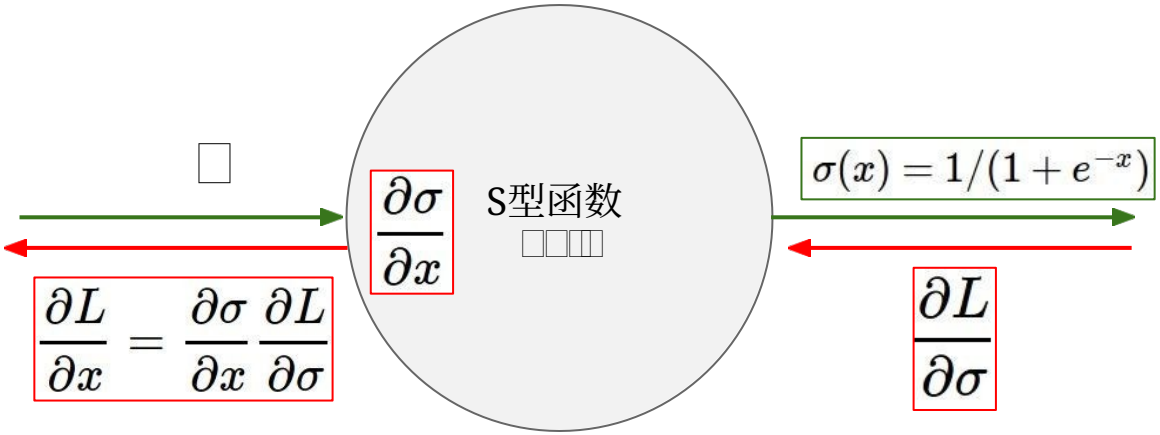
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

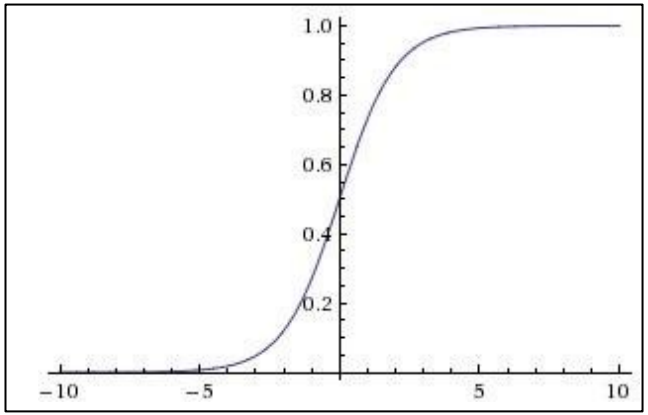




当 x ? -10 时会发生什么?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$



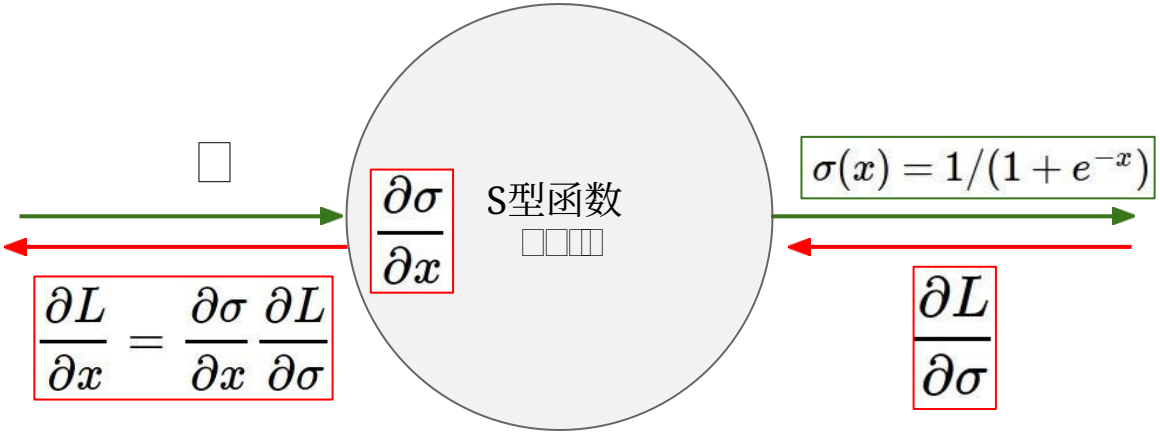


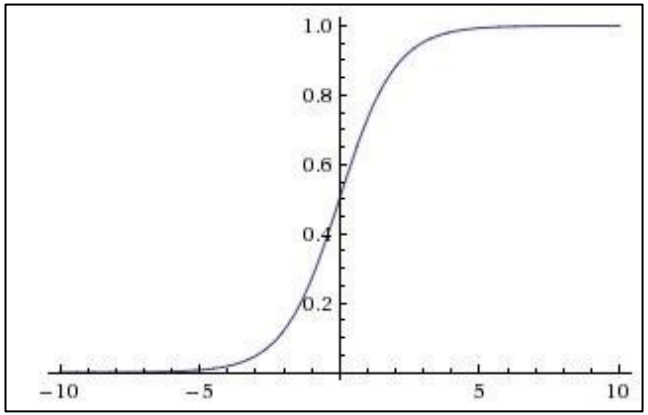
当 x ? -10 时会发生什么?

$$\sigma(x) = \sim 0$$

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)\,) = 0(1-0) = 0$$

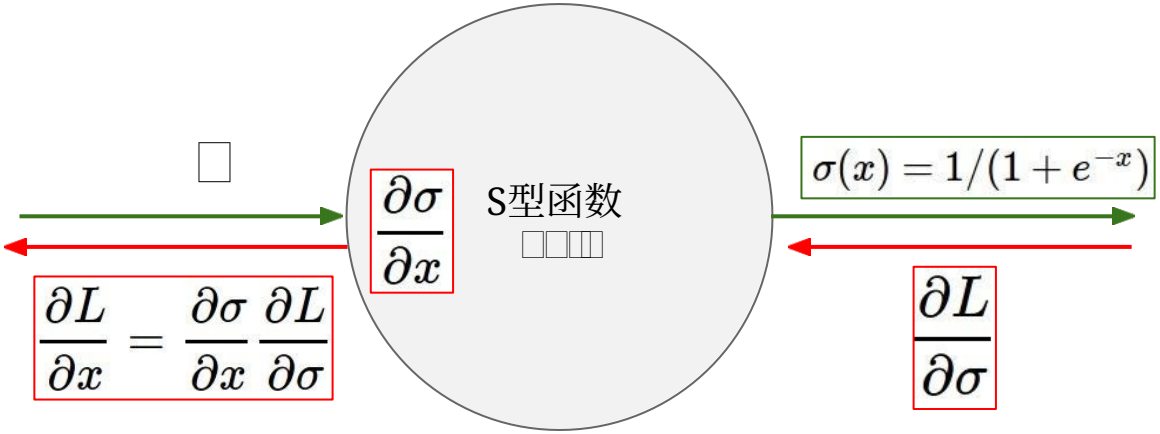
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)\,)$$

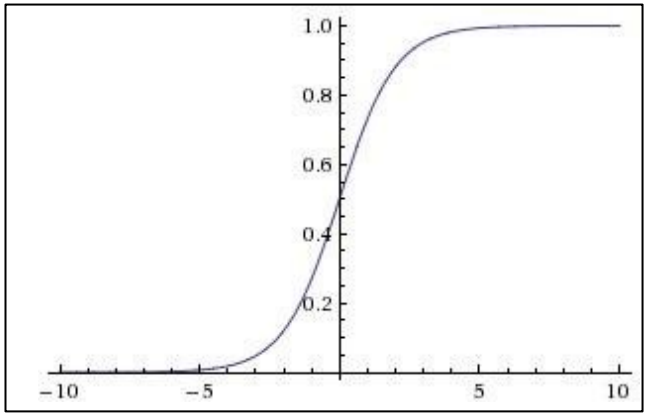




当 x ?-10?　当 x ? 0?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

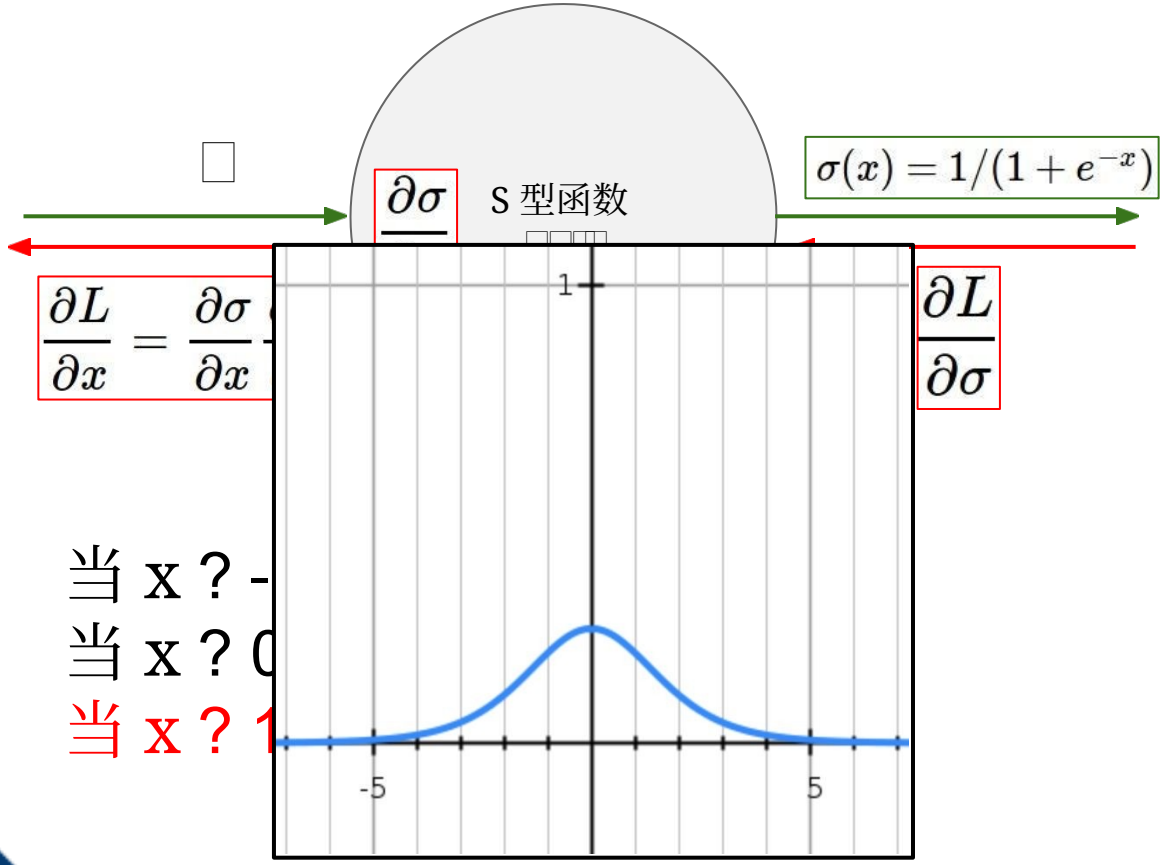


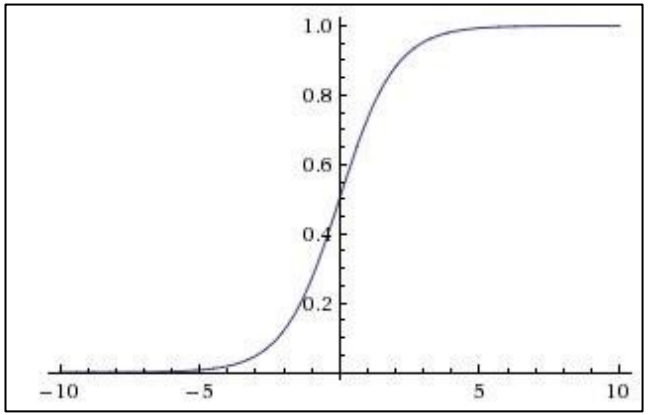


当 x ?-10? 当 x ? 0? 当 x ? 10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x))$$

$$\sigma(x) = \sim 1 \qquad \frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1-\sigma(x)) = 1(1-1) = 0$$





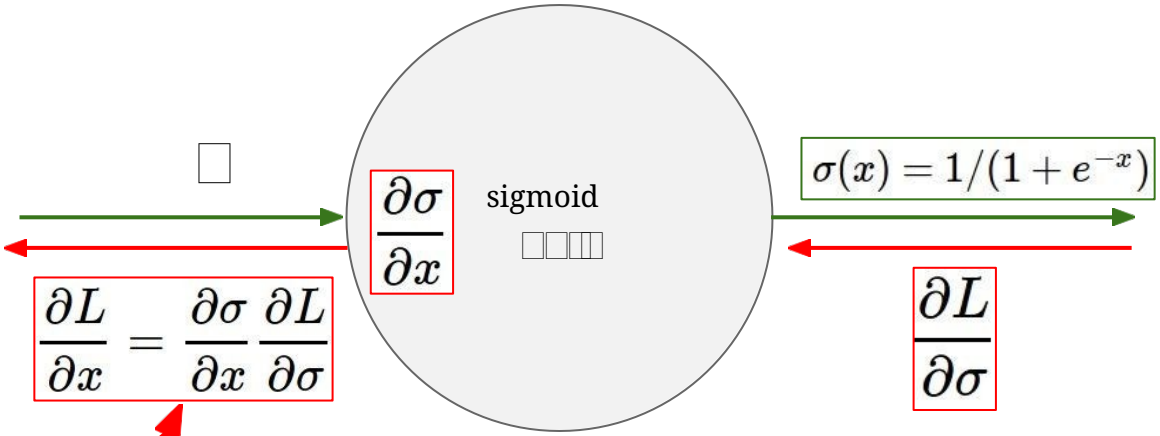
当 x ? -

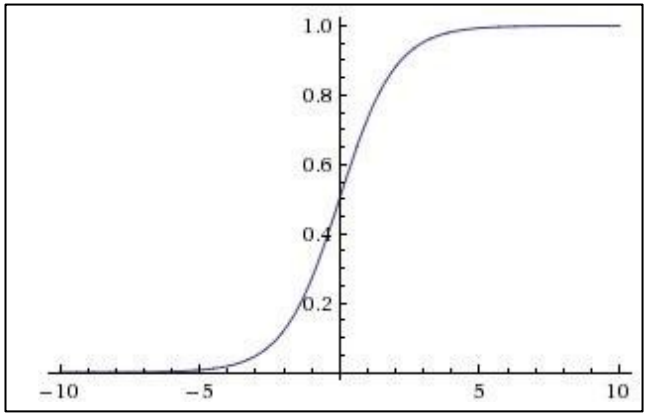
当 x ? 0

当 x ? 1

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

梯度



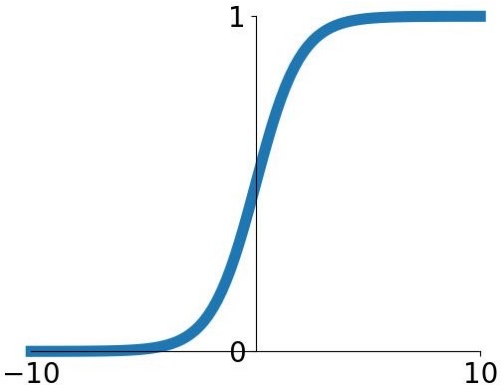


$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

为什么这是一个问题?

如果反向流动的所有梯度都为零，权重将永远不会改变

激活函数



Sigmoid

$$\sigma(x) = 1/(1+e^{-x})$$

- 将数值压缩到范围□0□□

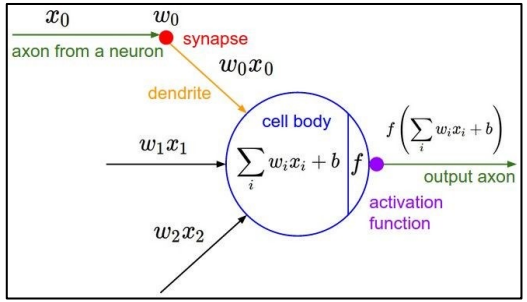
- 历史上很受欢迎，因为它们可以被很好地解释为神经元的饱和“发放率”

1. 饱和神经元会“杀死”梯度

2. Sigmoid 输出不是零中心的为什么这不好?

考虑当神经元的输入始终为正时会发生什么……

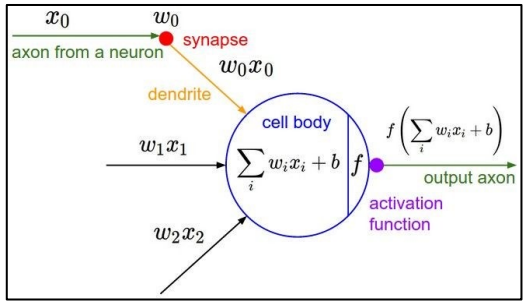
$$f\left(\sum_i w_i x_i + b\right)$$



关于 **w** 上的梯度，我们能说什么？

考虑当一个神经元的输入始终为正时会发生什么……

$$f\left(\sum_i w_i x_i + b\right)$$

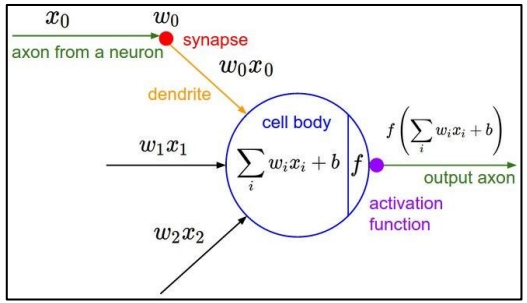


关于 **w** 上的梯度，我们能说什么？

$$\frac{\partial L}{\partial w} = \sigma(\sum_i w_i x_i + b)(1 - \sigma(\sum_i w_i x_i + b))x \times upstream_gradient$$

考虑当一个神经元的输入始终为正时会发生什么……

$$f\left(\sum_i w_i x_i + b\right)$$



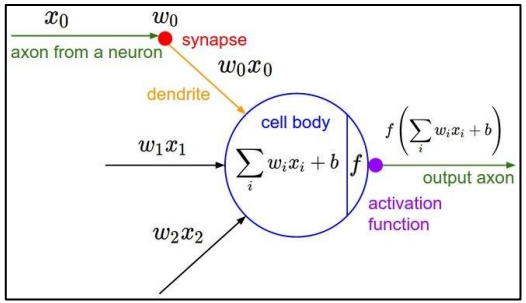
关于 **w** 的梯度，我们能说些什么？

我们知道 sigmoid 的局部梯度始终为正

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right) x \times upstream_gradient$$

考虑当一个神经元的输入始终为正时会发生什么……

$$f\left(\sum_i w_i x_i + b\right)$$



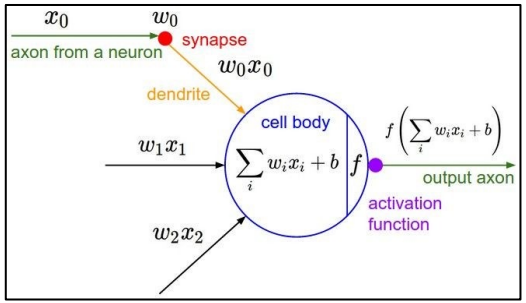
关于 **w** 的梯度，我们能说些什么?

我们知道 sigmoid 函数的局部梯度始终为正 我们假设 x 始终为正

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right)x \times upstream_gradient$$

考虑当神经元的输入始终为正时会发生什么……

$$f\left(\sum_i w_i x_i + b\right)$$



关于 **w** 的梯度，我们能说些什么?

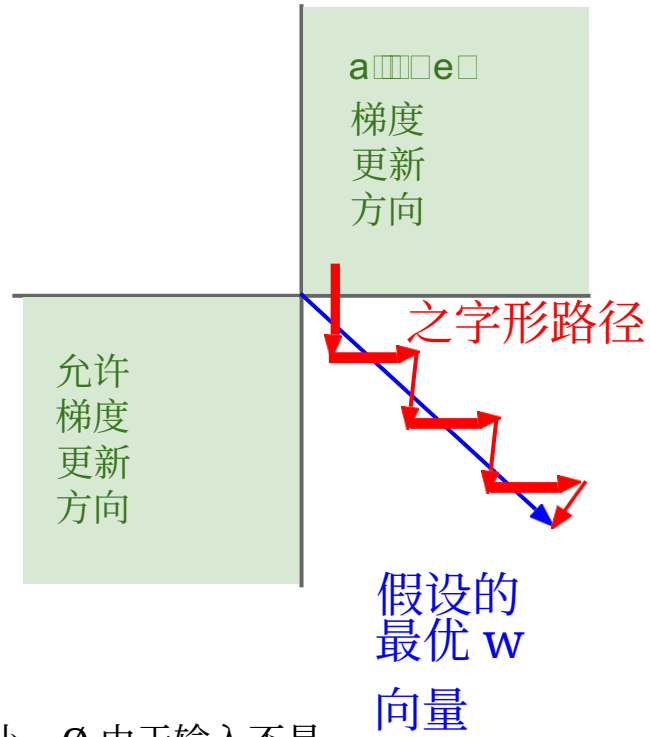
我们知道 sigmoid 的局部梯度始终为正 我们假设 x 始终为正

所以！！梯度的符号 **对于所有 w_i** 与上游标量梯度的符号相同 ！

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right)x \times upstream_gradient$$

考虑当神经元的输入始终为正时会发生什么……

$$f\left(\sum_i w_i x_i + b\right)$$



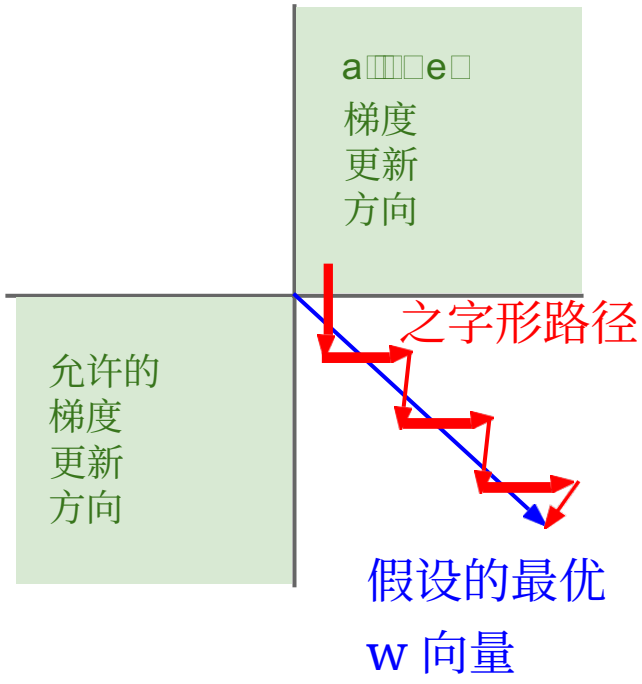
关于 **w** 上的梯度，我们能说些什么？

始终全为正或全为负

Ø 想象这样一个场景：最优解要求权重 w_1 增大，但权重 w_2 减小。Ø 由于输入不是零中心的，网络 **无法同时做出这些相反的调整**。它被迫在单步中要么增大所有权重，要么减小所有权重。

考虑当神经元的输入始终为正时会发生什么...

$$f\left(\sum_i w_i x_i + b\right)$$



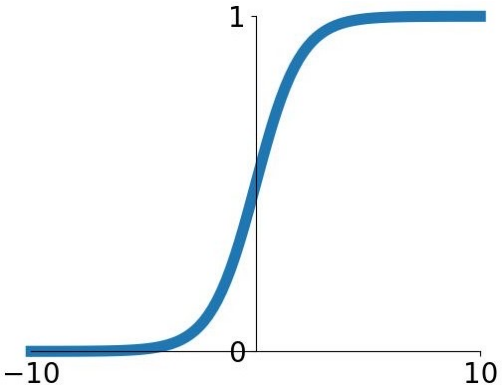
关于 **w** 的梯度，我们能说些什么?

总是全为正或全为负 :(

如何改进?

对于单个元素！小批量会有帮助

激活函数



Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

- 将数值压缩到范围 0

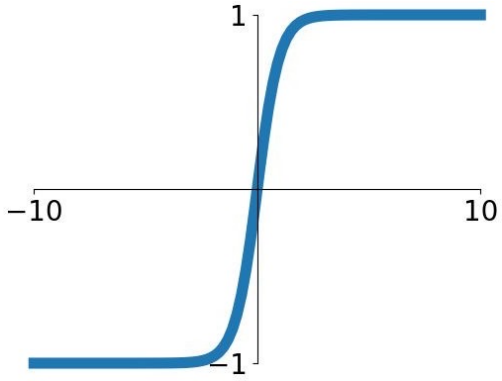
- 历史上很受欢迎，因为它们可以很好地解释为神经元的饱和“发放率”

1. 饱和神经元会“杀死”梯度

2. 输出不是零中心的



激活函数

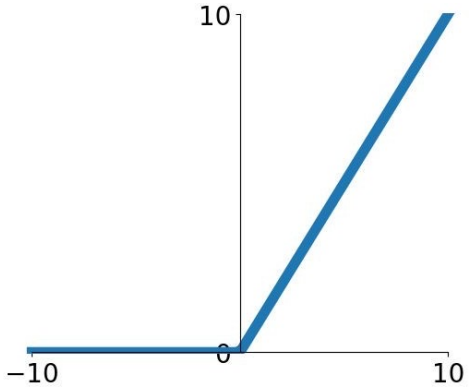


tanh(x)

- 将数值压缩到范围 z1 1 □- 零中心化（很好） - 饱和时仍会扼杀梯度:(

[LeCun et al., 6NN6□

激活函数



ReLU
（修正线性单元）

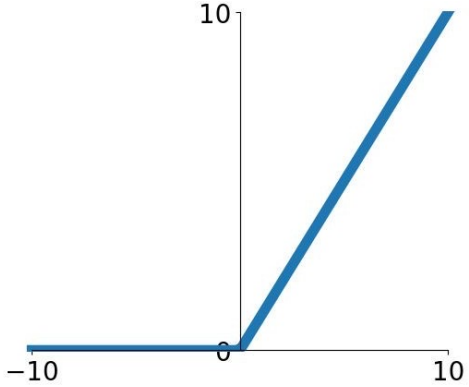
- 计算 **f(x) hmax(0,x)**

- 不会饱和（在 n区域） - 计算效率非常高 - 在实践中比 sigmoid/tanh 收敛快得多（例如6倍）

[Krizhevsky et al., 7567□



激活函数



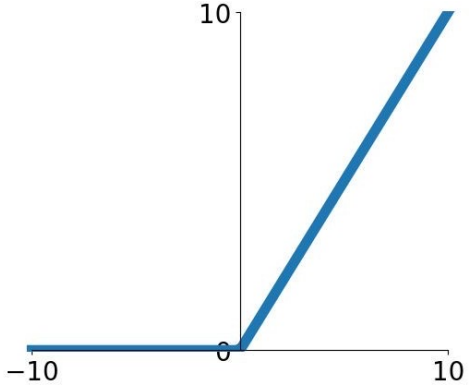
ReLU
（修正线性单元）

- 计算 **f(x) hmax(0,x)**

- 不会饱和（在 n区域） - 计算效率非常高 - 在实践中比 sigmoid/tanh 收敛快得多（例如 6 倍）

- 输出不以零为中心

激活函数

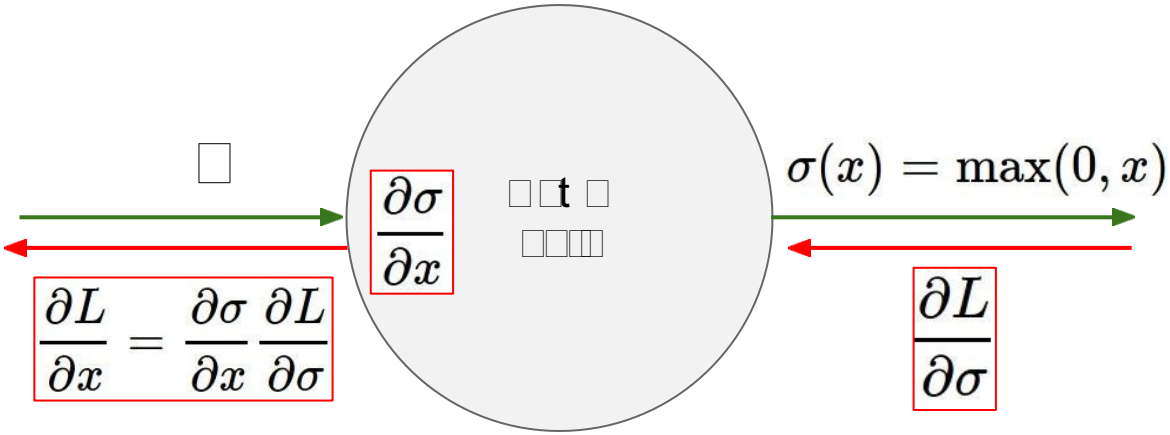


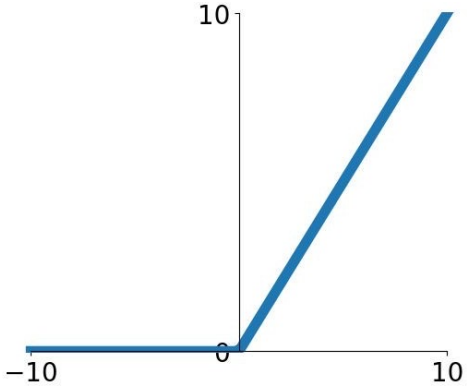
ReLU
（修正线性单元）

- 计算 **f(x) hmax(0,x)**

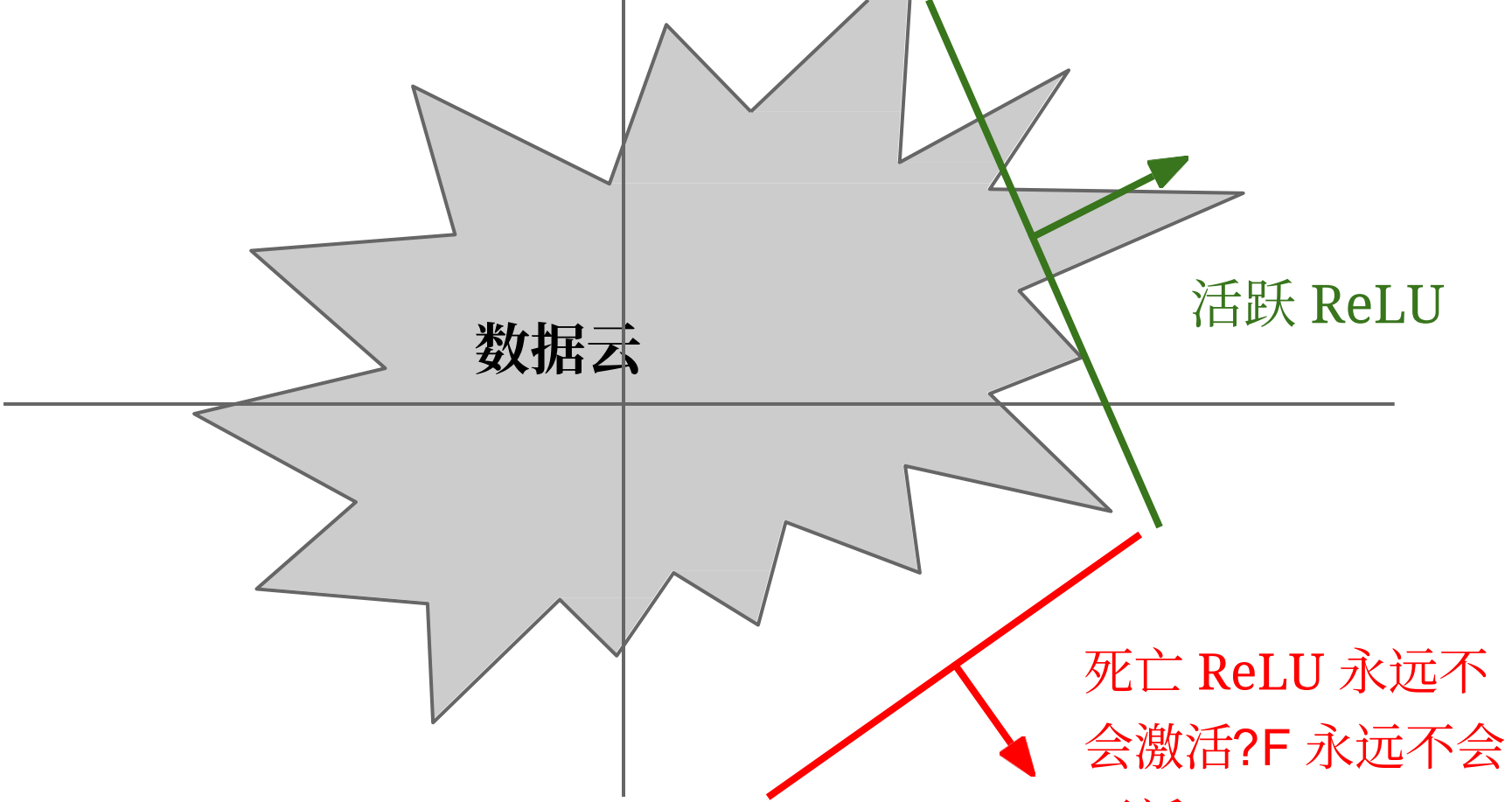
- 不会饱和（在 n区域） - 计算效率非常高 - 在实践中比 sigmoid/tanh 收敛快得多（例如 6 倍）

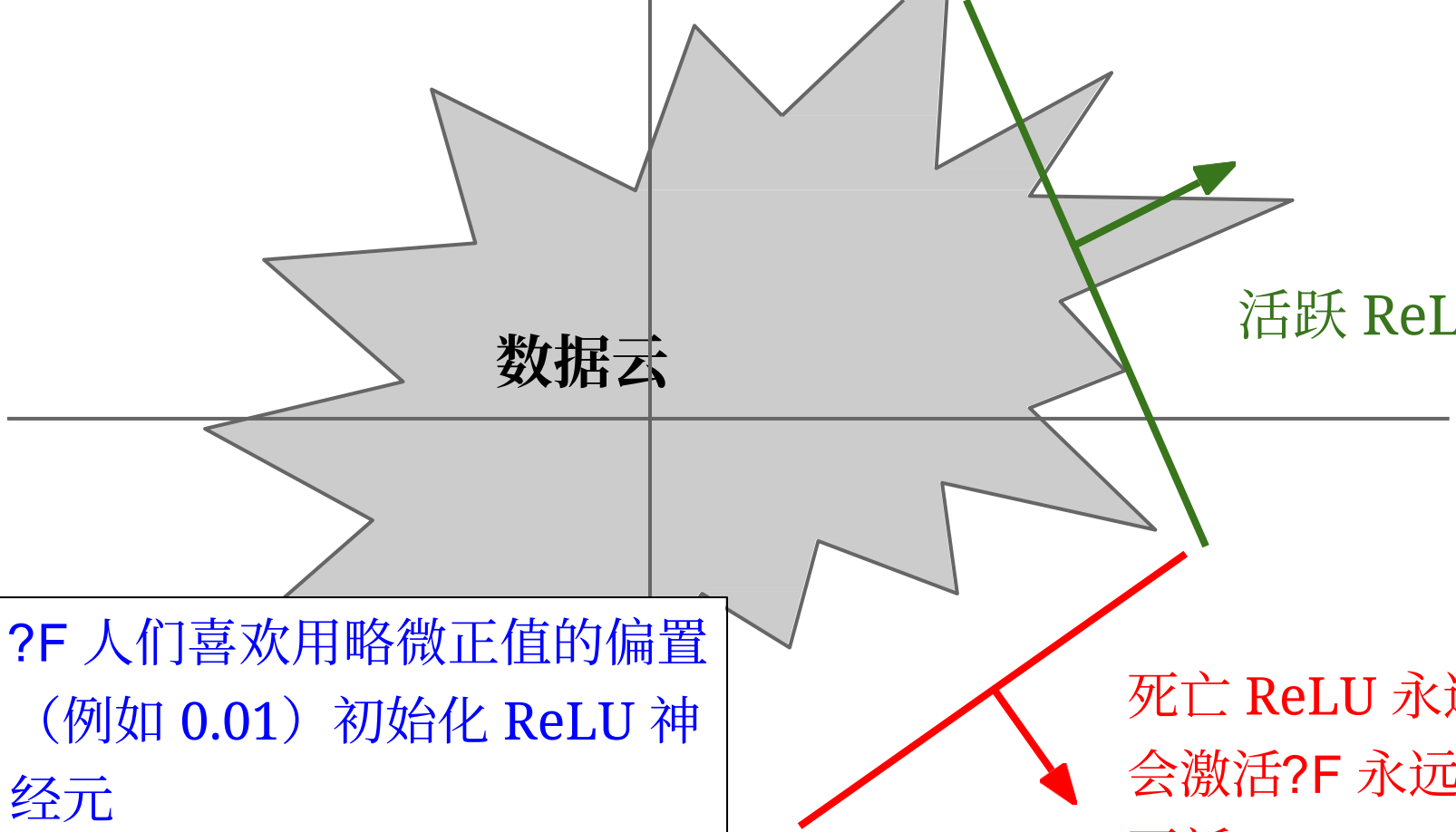
- 输出非零中心化 - 一个恼人的问题：死亡神经元





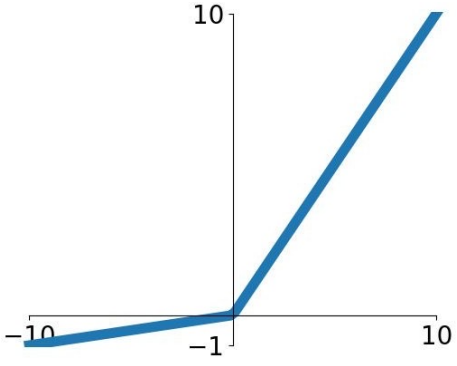
当 x ?-10 时会发生什么？当 x ? 0 时会发生什么？当 x ? 10 时会发生什么?


数据云
活跃 ReLU
死亡 ReLU 永远不会激活?F 永远不会更新



?F 人们喜欢用略微正值的偏置（例如 0.01）初始化 ReLU 神经元

激活函数

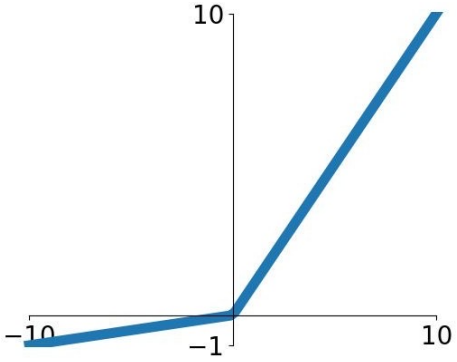


泄漏型 ReLU

$$f(x) = \max(0.01x, x)$$

- 不会饱和 - 计算效率高 - 在实践中比 sigmoid/tanh 收敛快得多！（例如 6x） - **不会“死亡”**。

激活函数



- 不会饱和 - 计算高效 - 在实践中比 sigmoid/tanh 收敛快得多！（例如 6 倍） - **不会“死亡”**。

泄漏型 ReLU

$$f(x) = \max(0.01x, x)$$

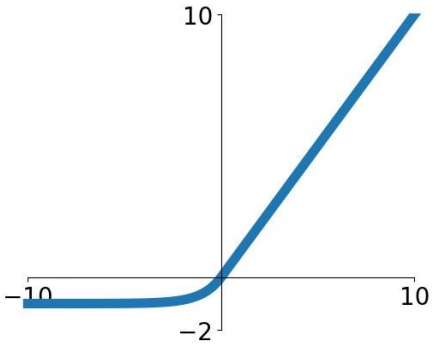
参数化整流器（PReLU）

$$f(x) = \max(\alpha x, x)$$

反向传播到 \alpha （参数）

激活函数

指数线性单元（ELU）



- 具备 ReLU 的所有优点 - 输出均值更接近零 - 与 Leaky ReLU 相比，负饱和区域增强了对噪声的鲁棒性

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\left(\exp(x) - 1\right) & \text{if } x \leq 0 \end{cases}$$

- 计算需要 exp()

(Alpha 默认值 URL)

Maxout“神经元”

- 不具备点积的基本形式 ->逐元素非线性

- 泛化了 ReLU 与 Leaky ReLU
- 线性区域。不会饱和。不会死亡。

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

问题：参数/神经元数量翻倍 :(



在实践中：

- 使用 Remu。注意你的学习率 es
- 尝试 mea□y Remu 2n a□out 2h mu 以挤出一些边际收益 - 不要使用 si□moid 或 tanh 在隐藏层中- 它们可用于输出层以控制 trol

输出范围



卷积



上一页：图像分类的线性模型



$I \quad Ne^{32\times32\times3}$

$s \quad Ne^{3072\times1}$

向量化

$W(?, b, x)$

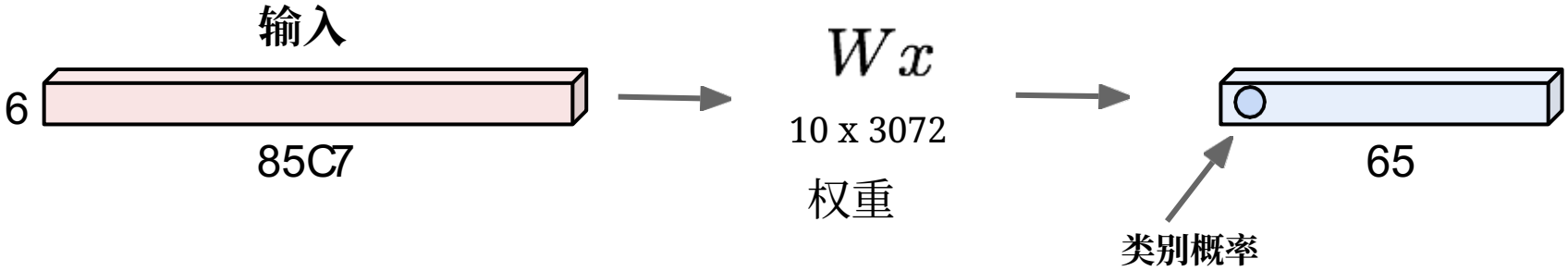
W, b

参数

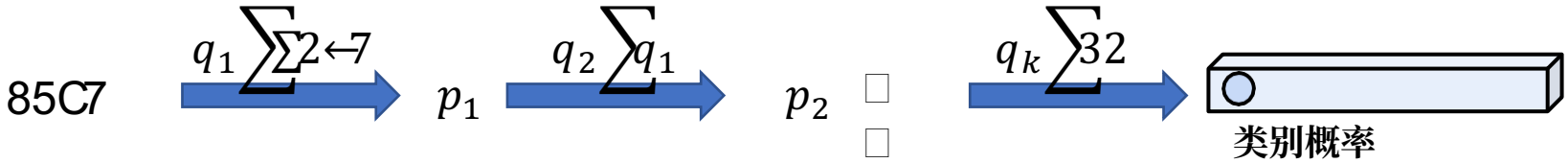
21个数字，给出类别得分

预测：最大得分

上一节：图像分类的线性模型

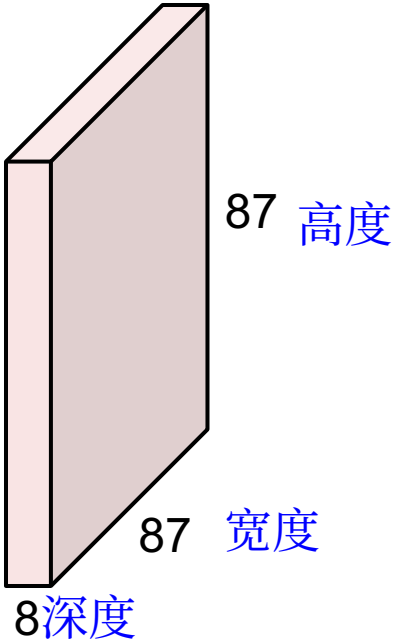


堆叠多个层以形成多层感知机：

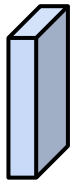


模型需要学习大量参数

卷积层



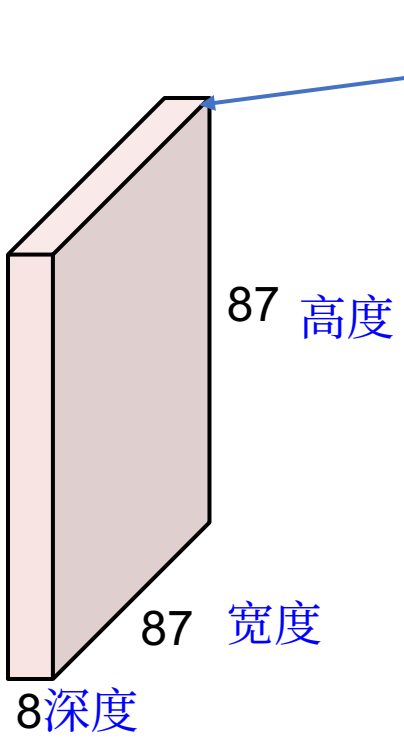
5x5x3 滤波器



将滤波器与图像进行**卷积**，即“在空间上滑过图像，计算点积”

参数数量：$5\times5\times3 = 45$

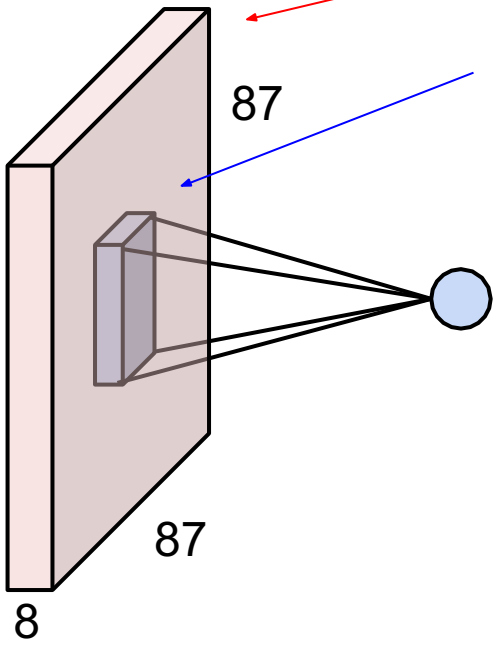
卷积层



将滤波器与图像进行**卷积**，即“在图像上空间滑动，计算点积

参数数量：$5\times5\times3 = 45$

卷积层



32x32x3 图像

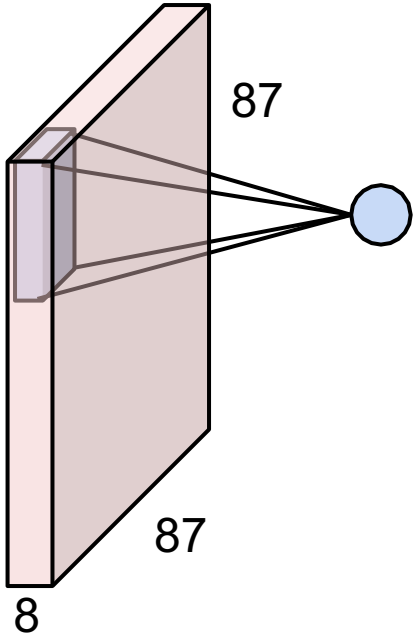
5x5x3 滤波器w

1 个数值：

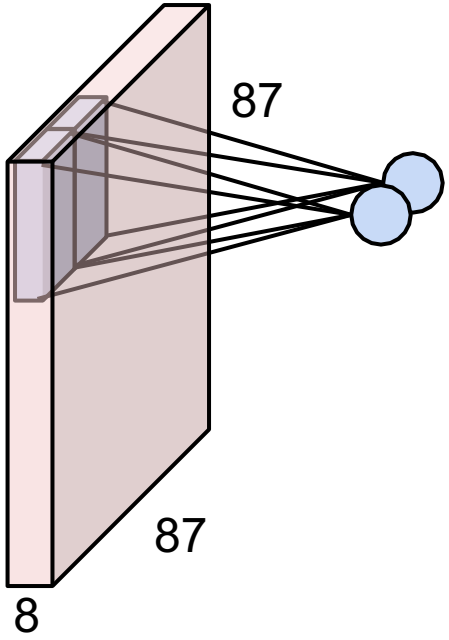
将滤波器与图像中一个小的 5x5x3 块进行点积所得到的结果（即 5*5*8 RC= 维点积 +偏置）

$$w^T x + b$$

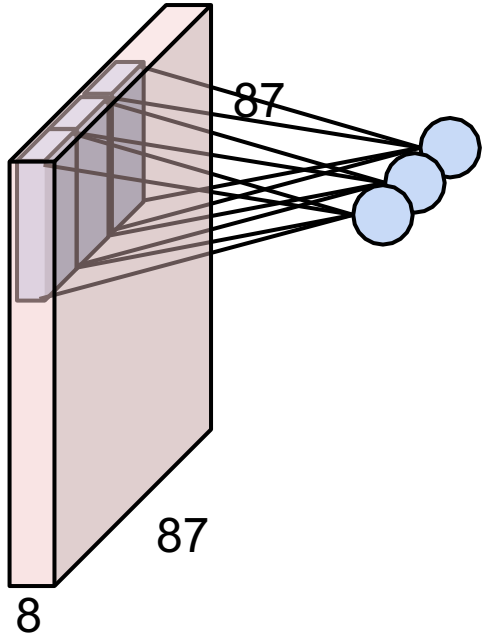
卷积层



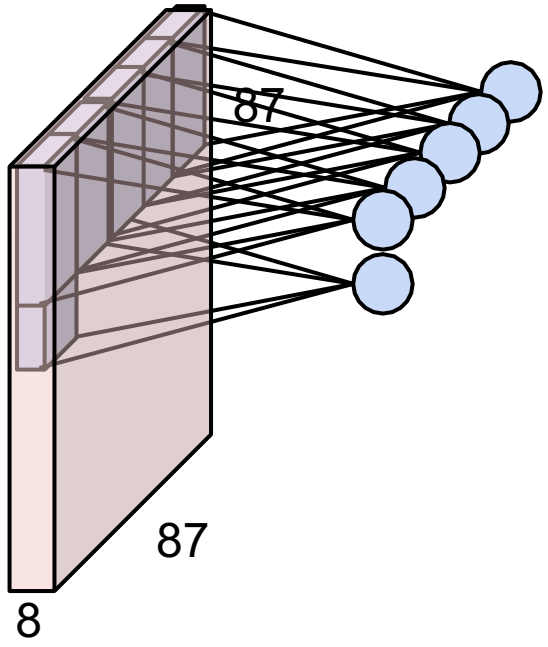
卷积层



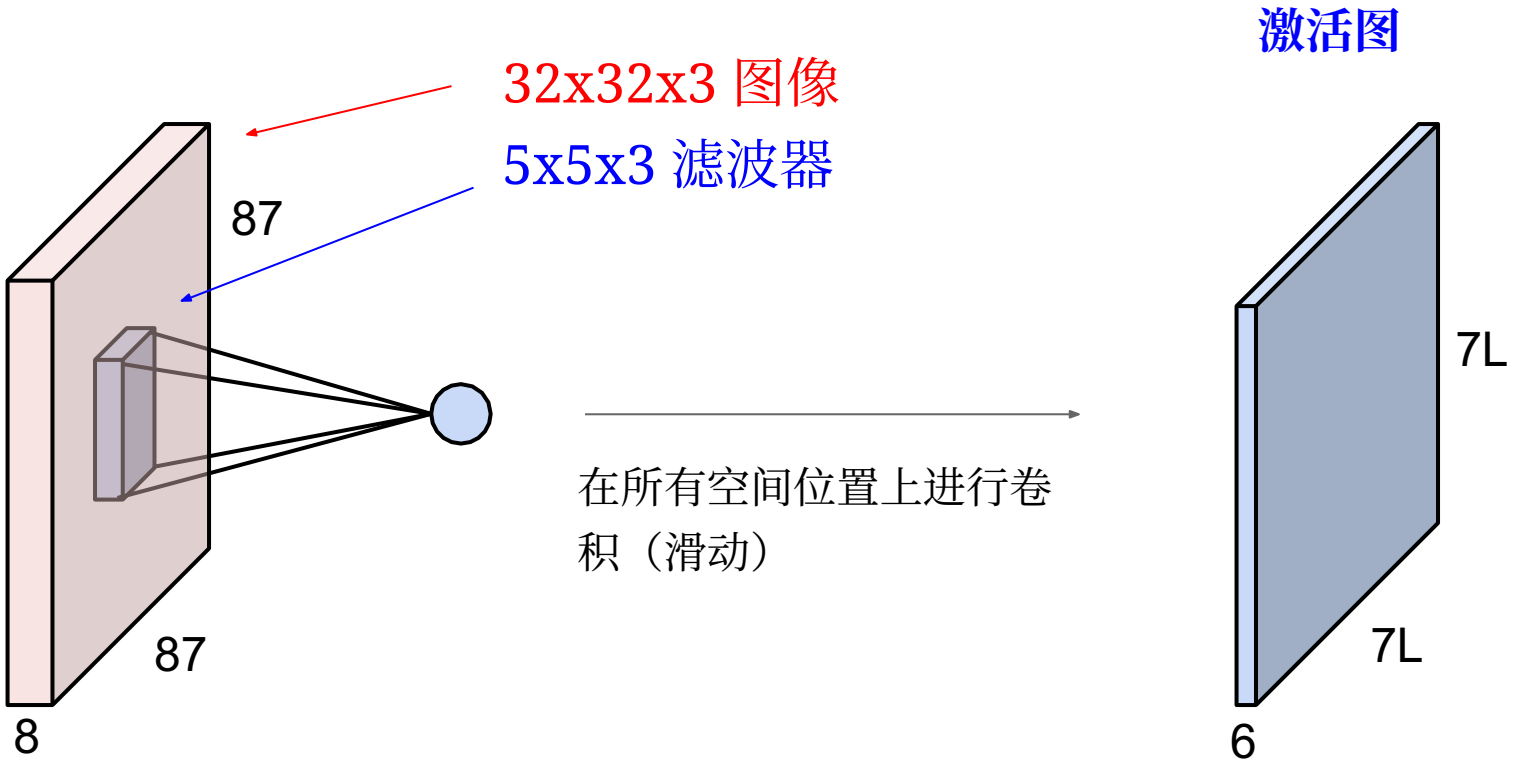
卷积层



卷积层

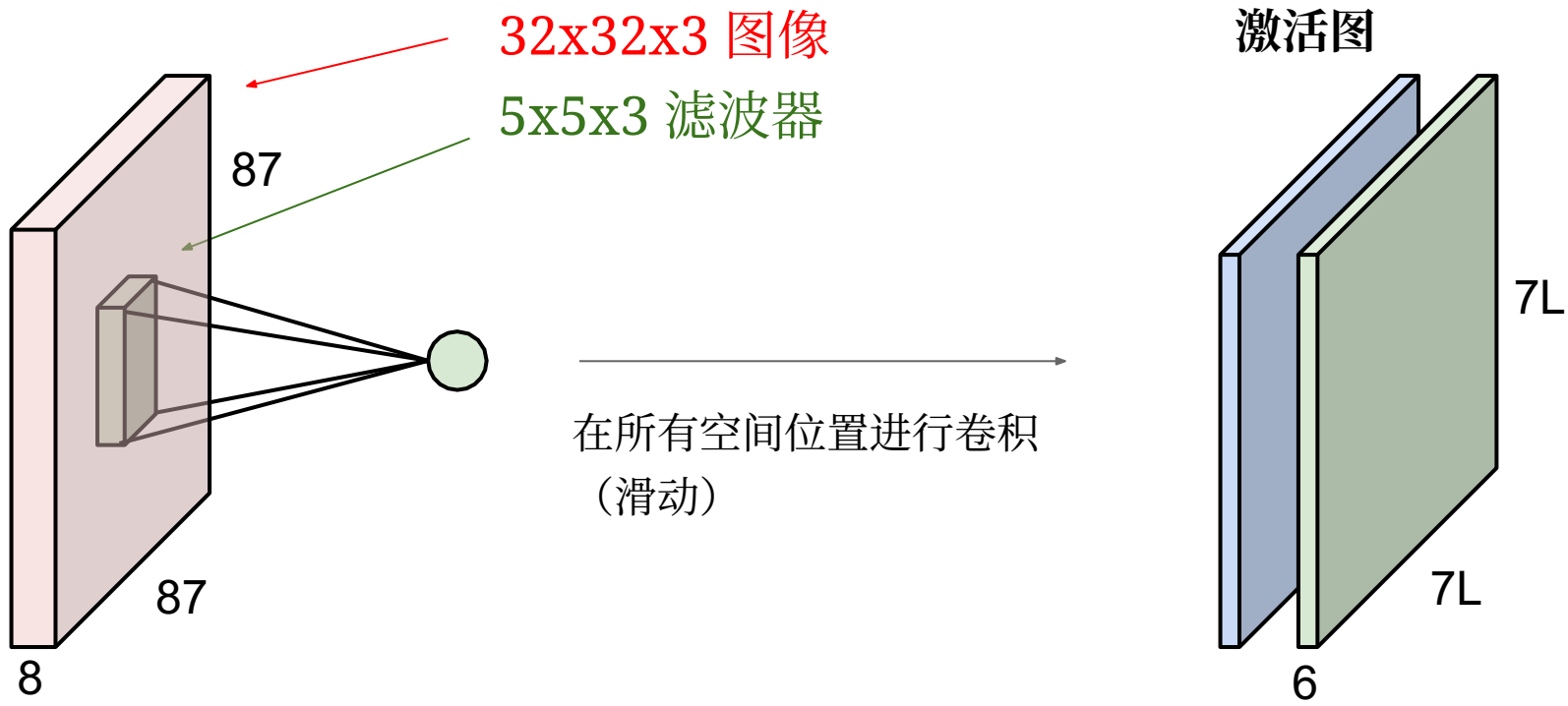


卷积层

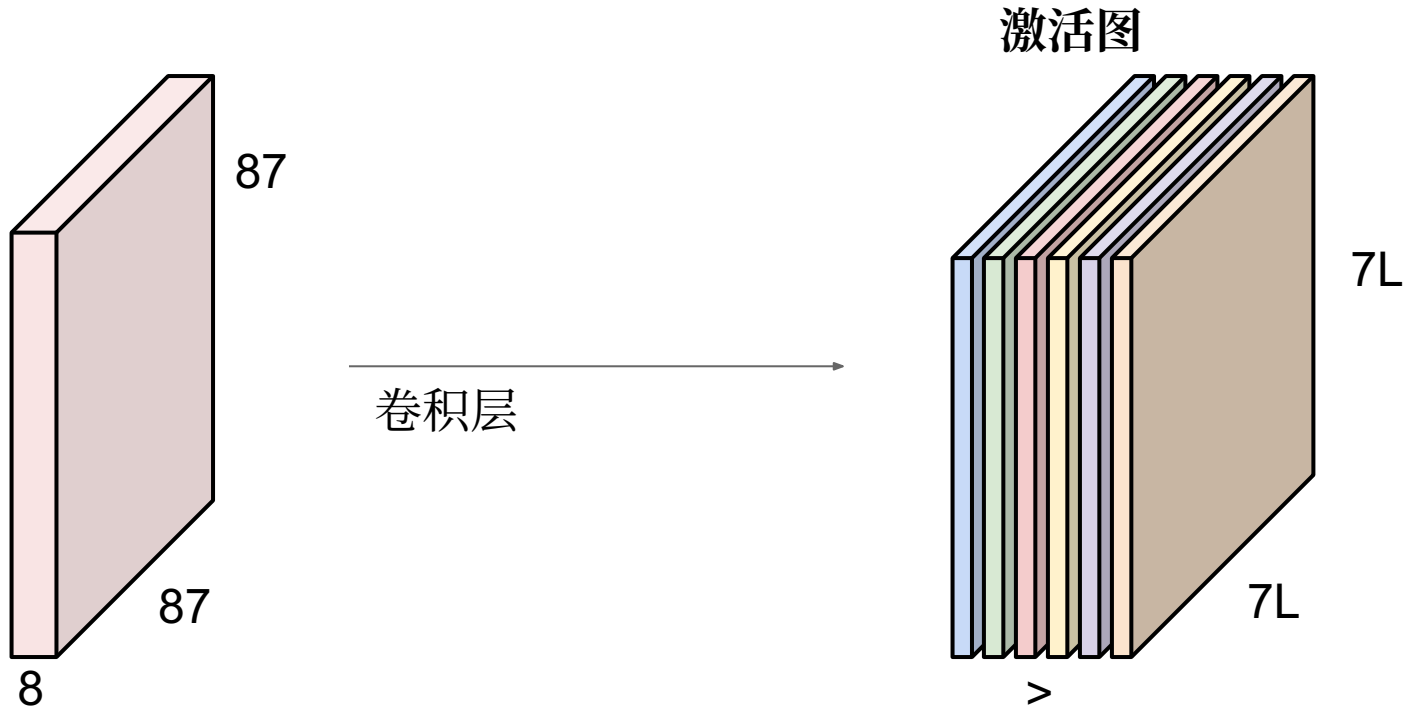


卷积层

考虑第二个绿色滤波器

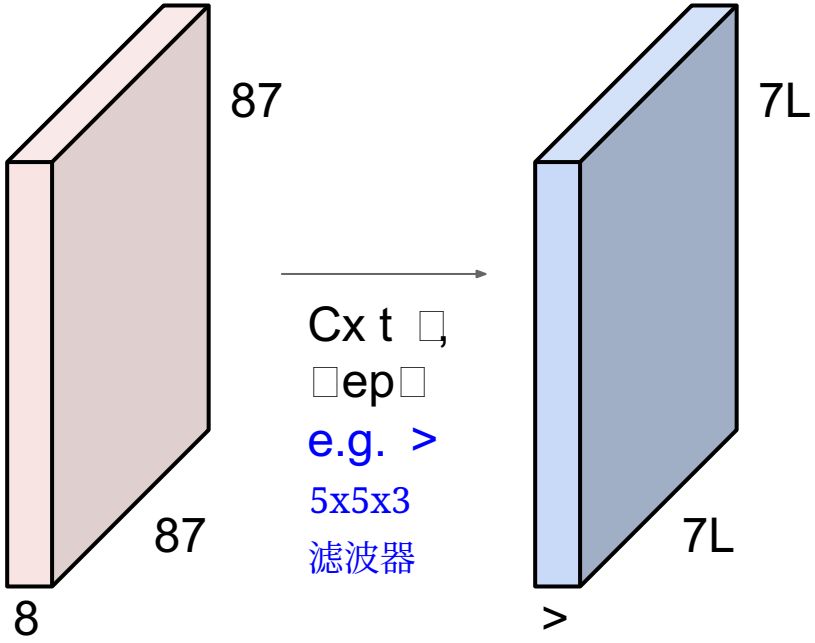


例如，如果我们有 6 个 5×5 滤波器，我们将得到 6 个单独的激活图：

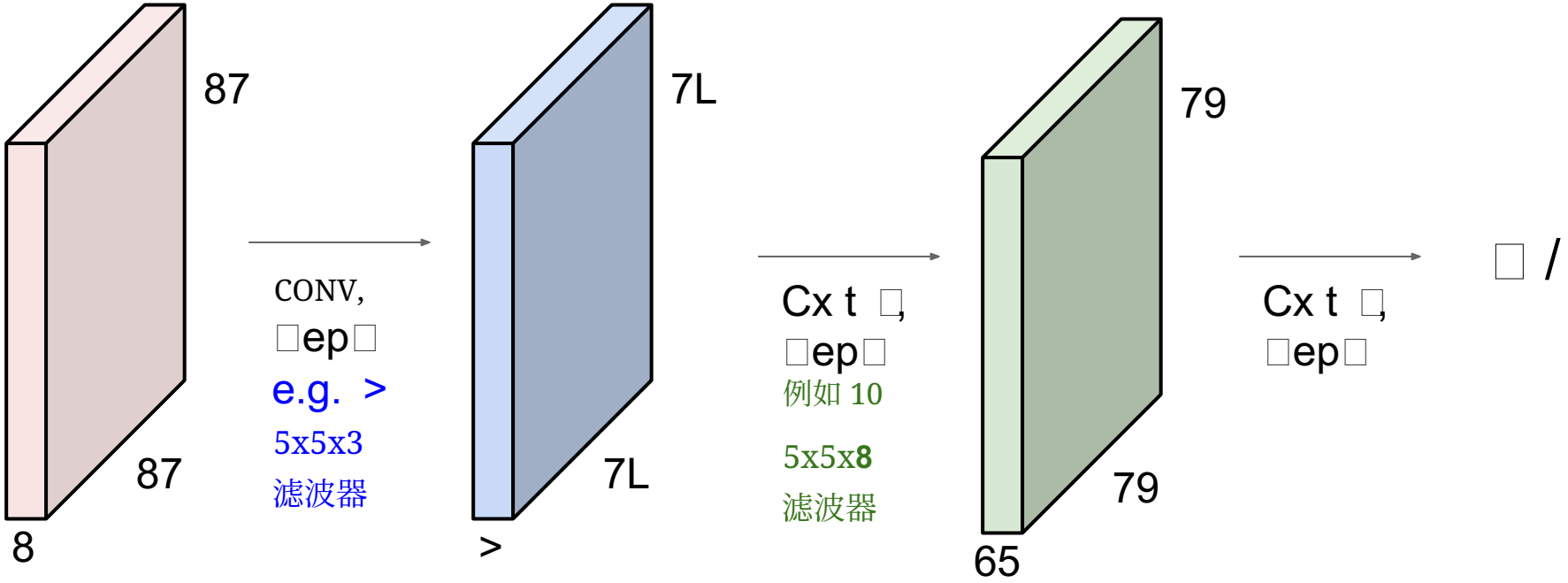


w 我们将这些堆叠起来，得到一张大小为 28x28x6 的“新图像”！

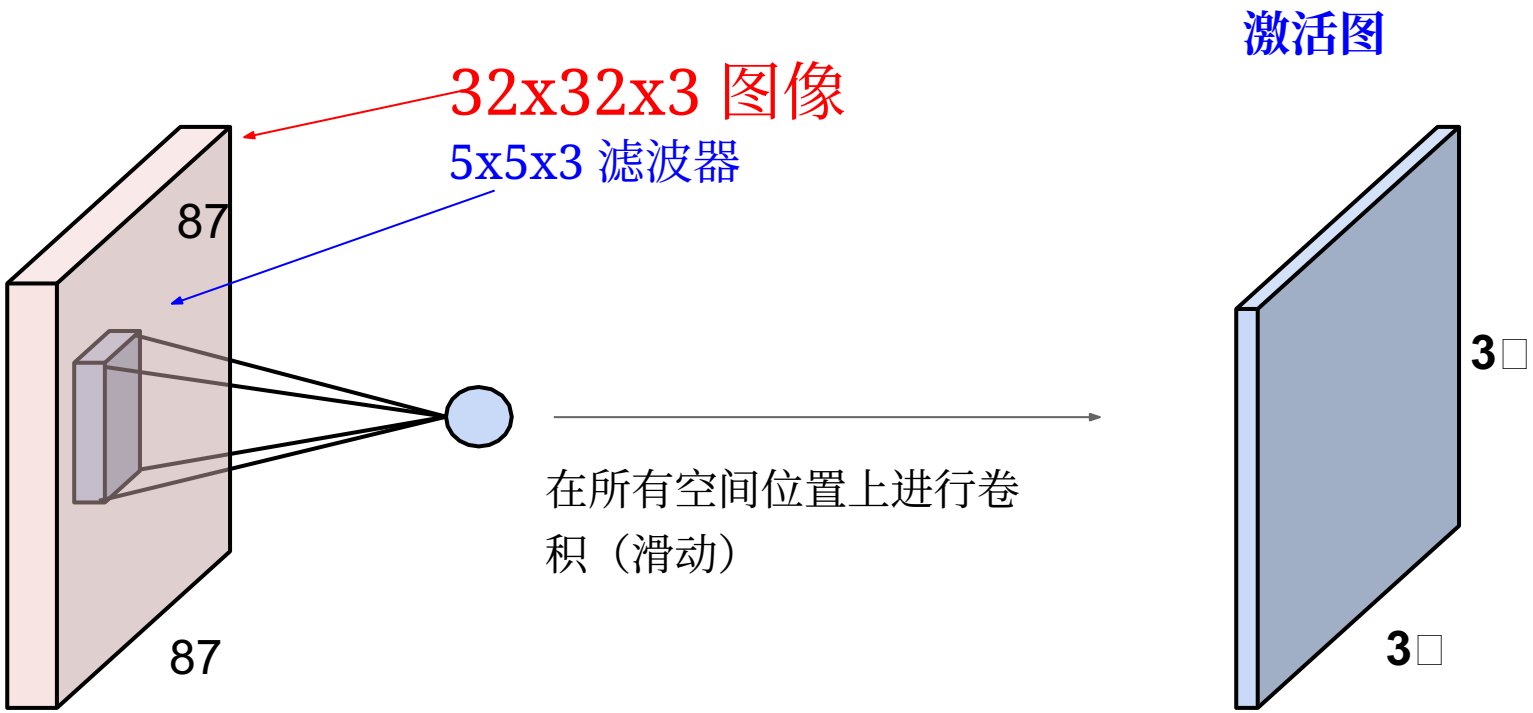
预览：卷积网络是一系列卷积层，其间穿插着激活函数



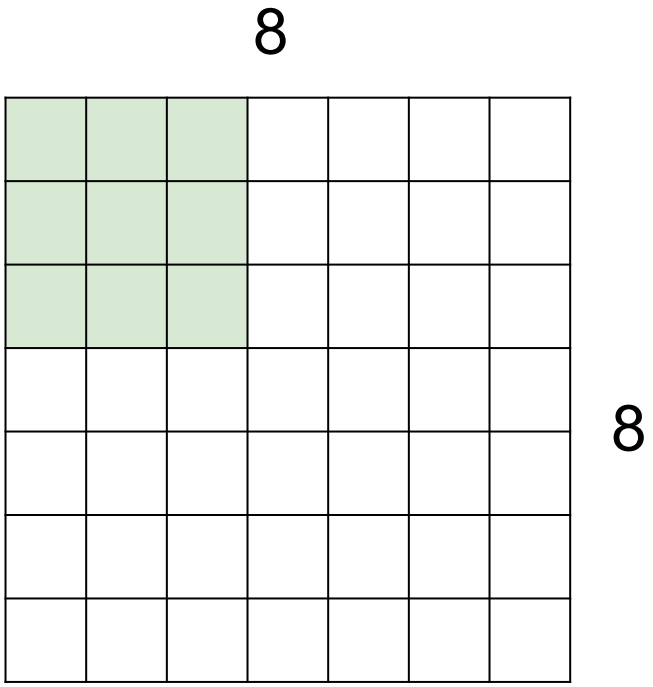
ConvNet 是一系列卷积层，其间穿插着激活函数



更深入地审视空间维度：



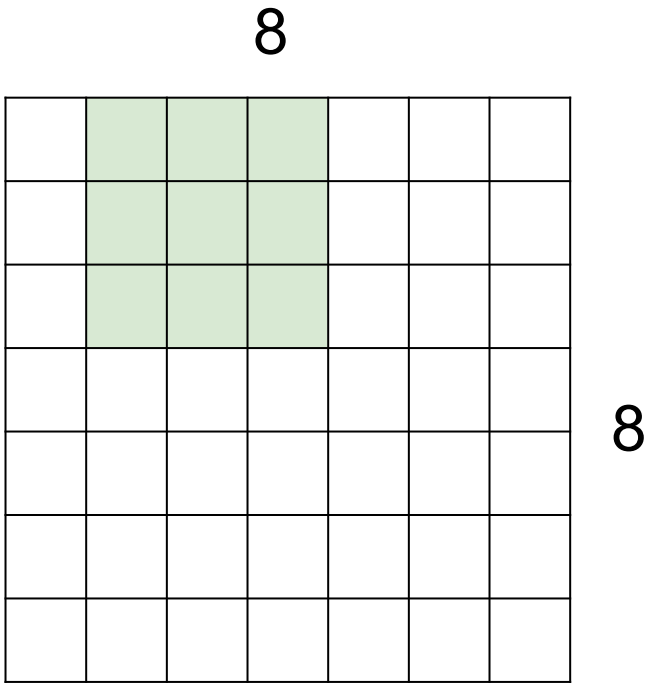
更深入地了解空间维度:



7x7 输入（空间上）假设

3x3 滤波器

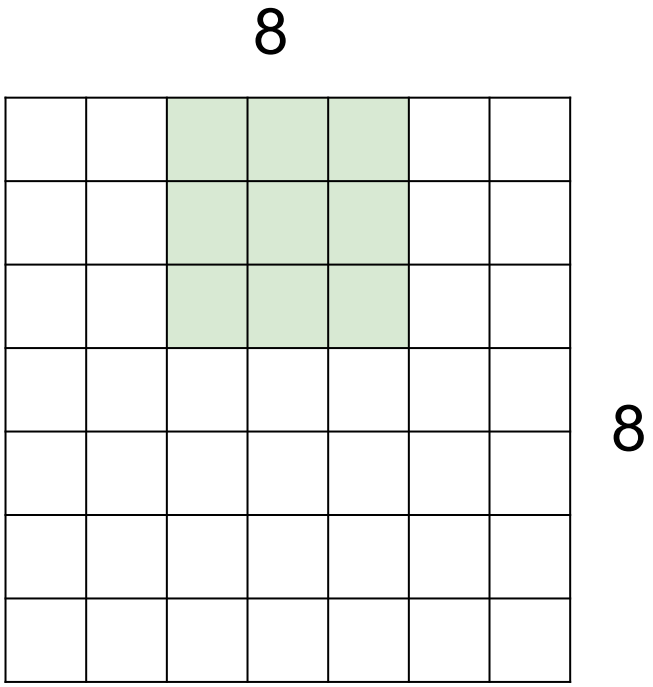
深入了解空间维度:



7x7 输入（空间上）假设

3x3 滤波器

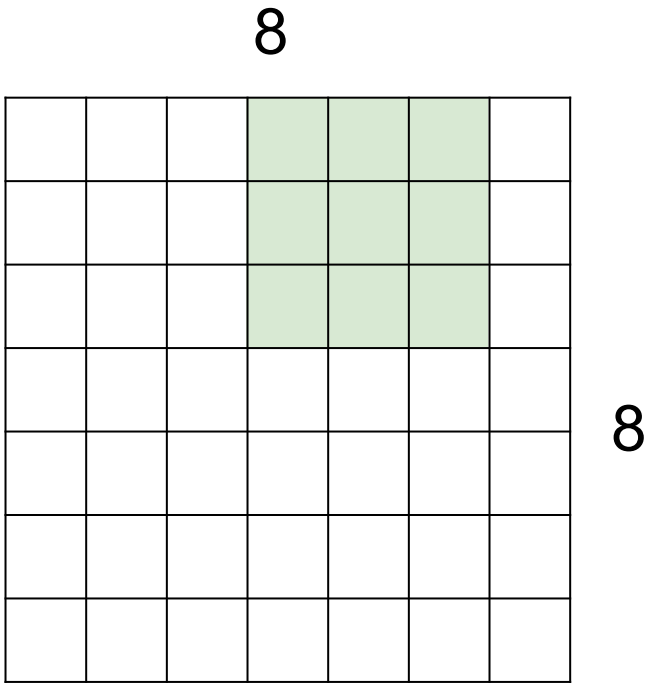
更深入地观察空间维度:



7x7 输入（空间上）假设

3x3 滤波器

深入了解空间维度:

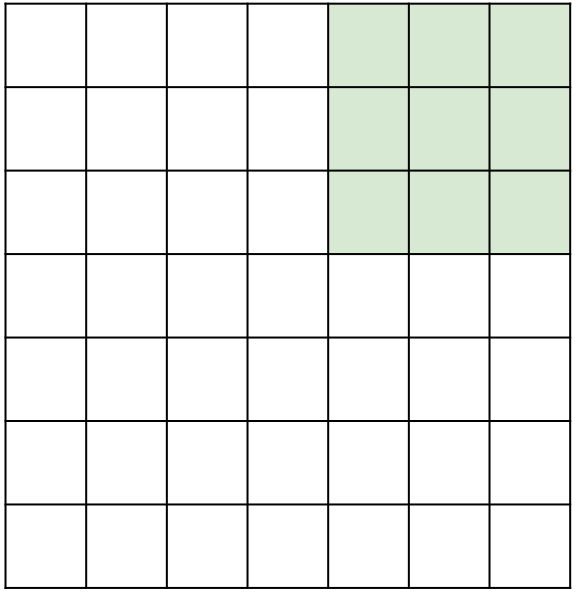


7x7 输入（空间上）假设

3x3 滤波器

深入探讨空间维度:

8



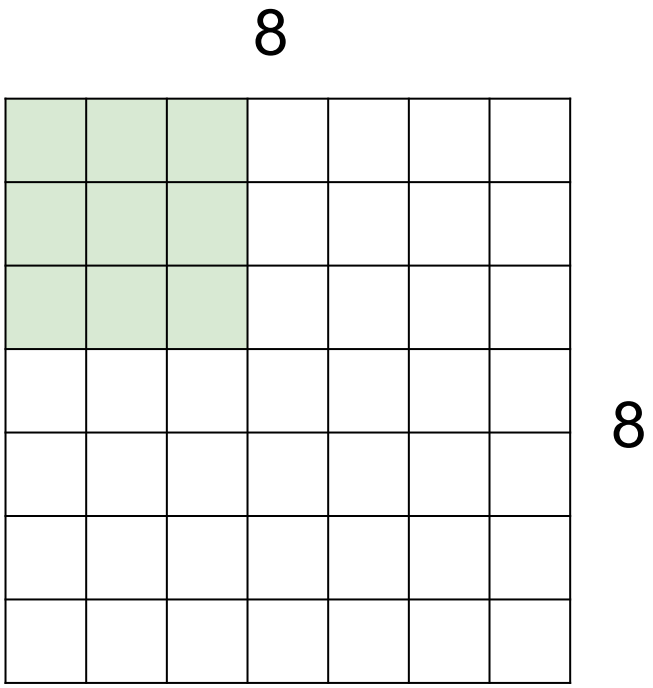
8

7x7 输入（空间上）假设

3x3 滤波器

hn ex5 输出

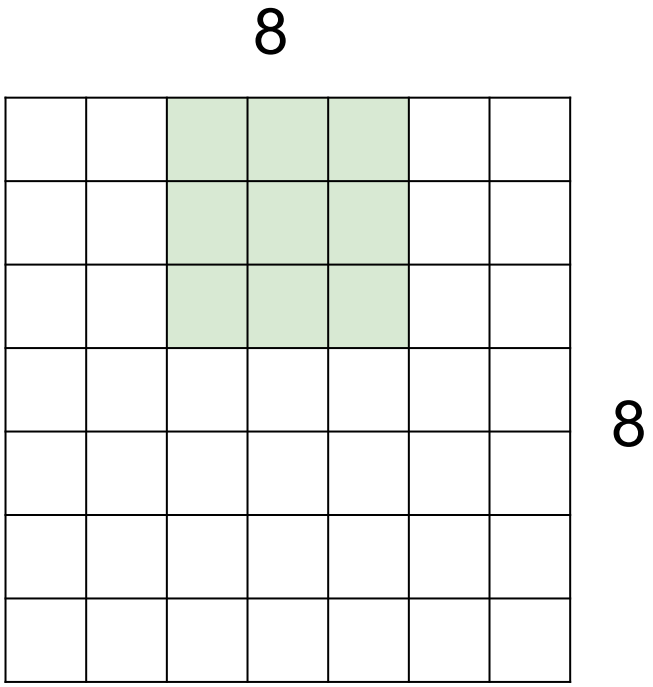
深入了解空间维度:



7x7 输入（空间上）假设应用 3x3 滤波器，

步幅为 2

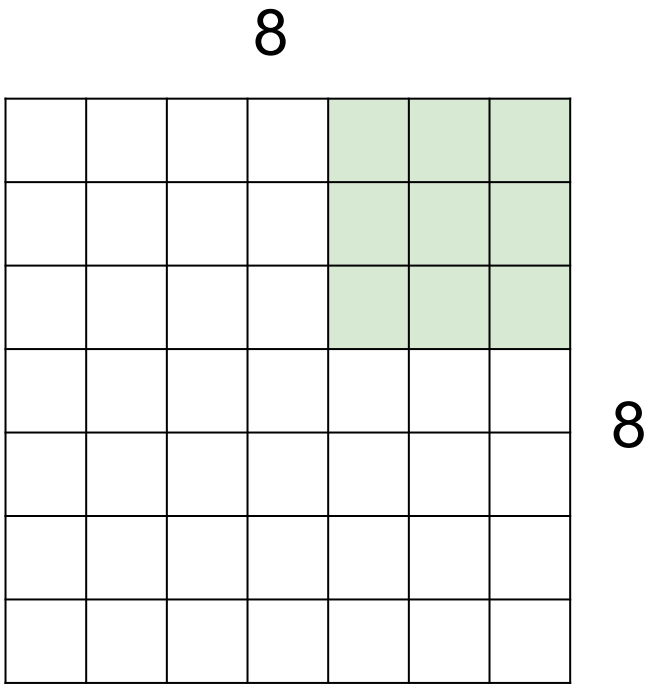
深入了解空间维度:



7x7 输入（空间上）假设应用 3x3 滤波器，

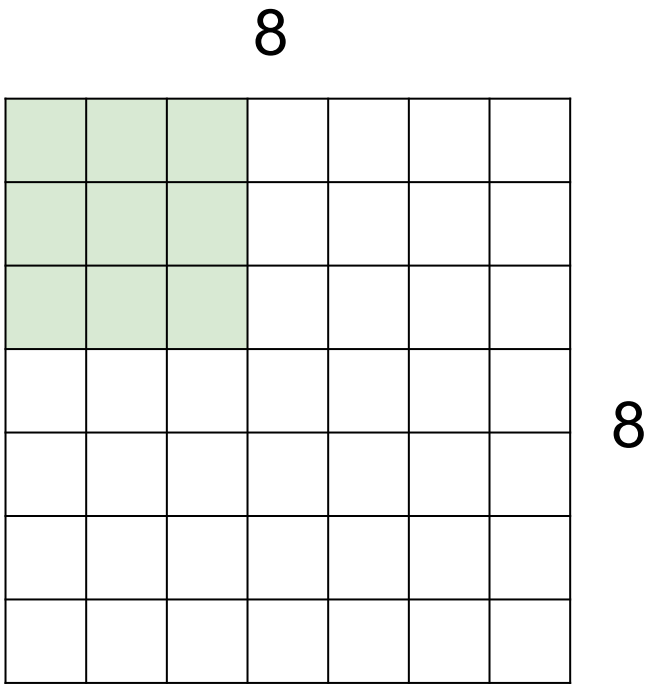
步幅为 2

深入了解空间维度：



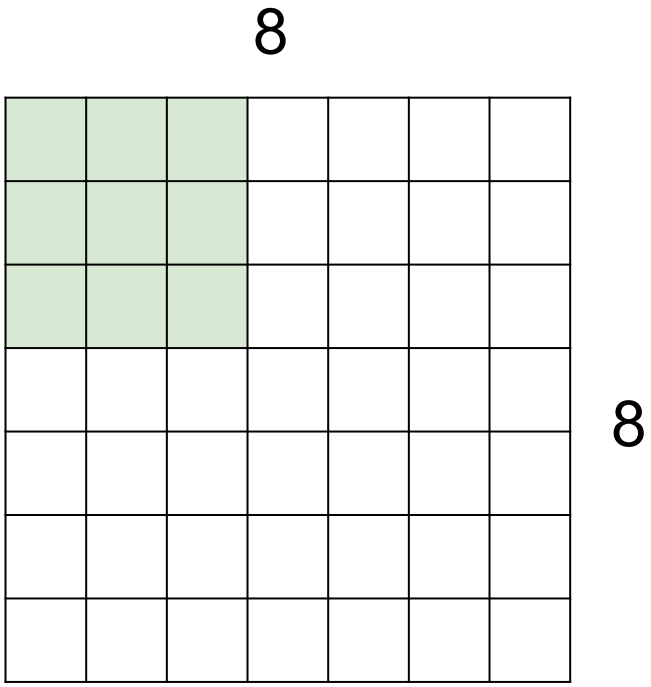
7x7 输入（空间上）假设应
用 3x3 滤波器 **使用步幅**
hn a a x 输出！

深入了解空间维度：



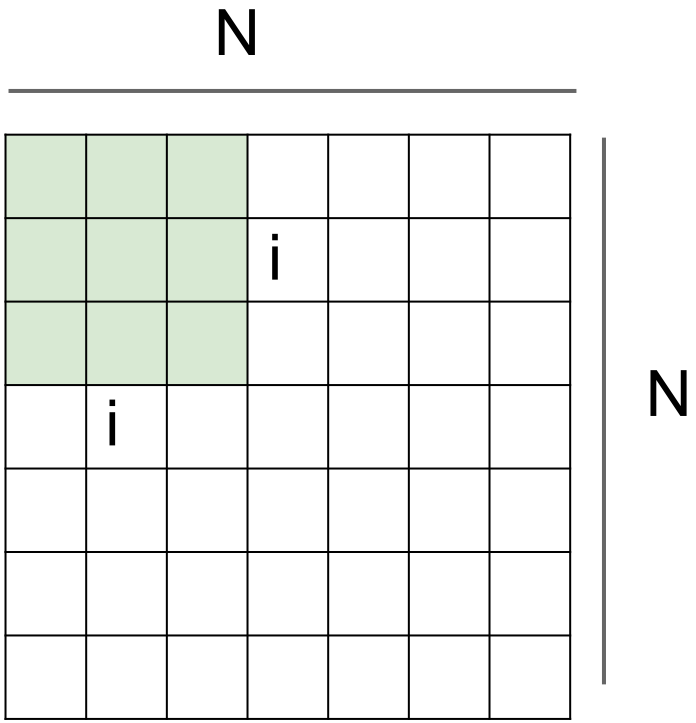
7x7 输入（空间上）假设应用 3x3 滤波器 **使用步幅 3?**

深入了解空间维度：



7x7 输入（空间上）假设应用 3x3 滤波器 **步幅为 3?**

不匹配！ 无法以步长 3 在 7x7 输入上应用 3x3 滤波器。

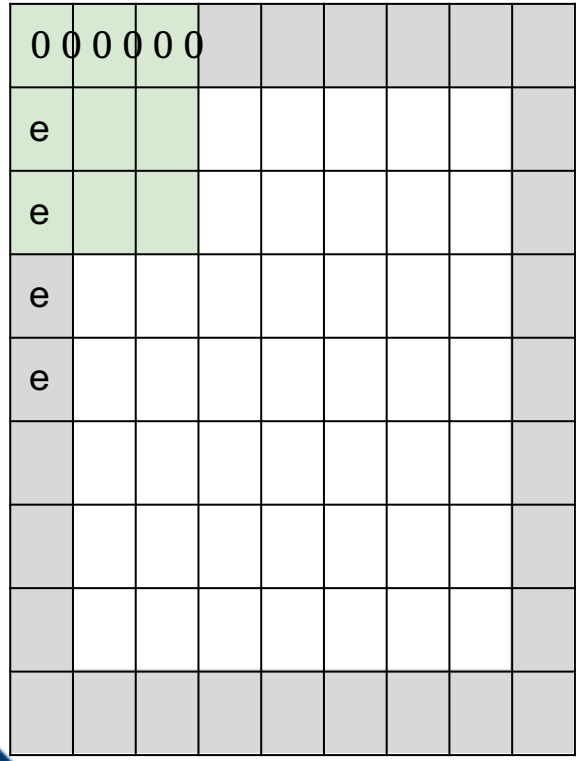


输出尺寸：**(N -**

F) / stride 3 R

例如 N？8，F？4：步长 1 ?F (7 - 3)/1，1？6 步长 3 ?F (7 - 3)/3，1？4 步长 4 ?F (7 - 3)/4，1？3.33

实践中：通常对边界进行零填充

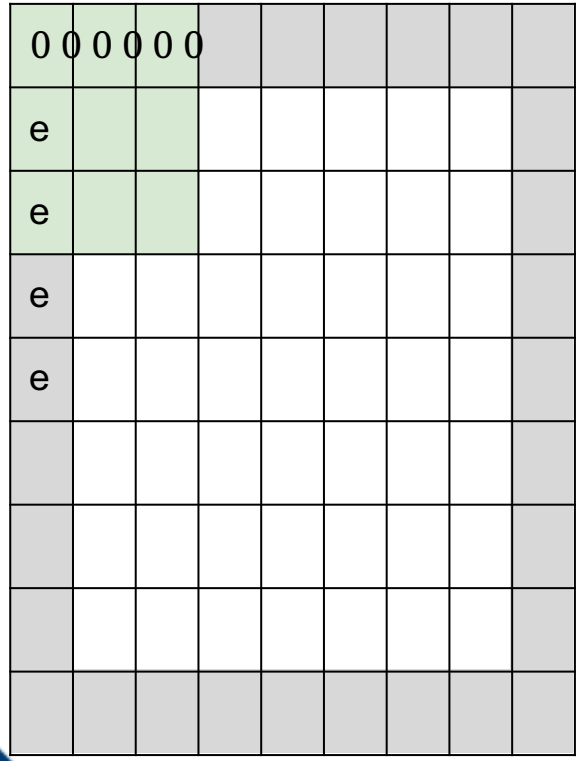


例如 输入 7 7 □□ x□滤波器，应用于 **步长 1 填充 1 像素** 边界 de 输出是什么?

（回顾：） (N -

i) 0stri□e ， 1

在实践中：通常对边界进行零填充



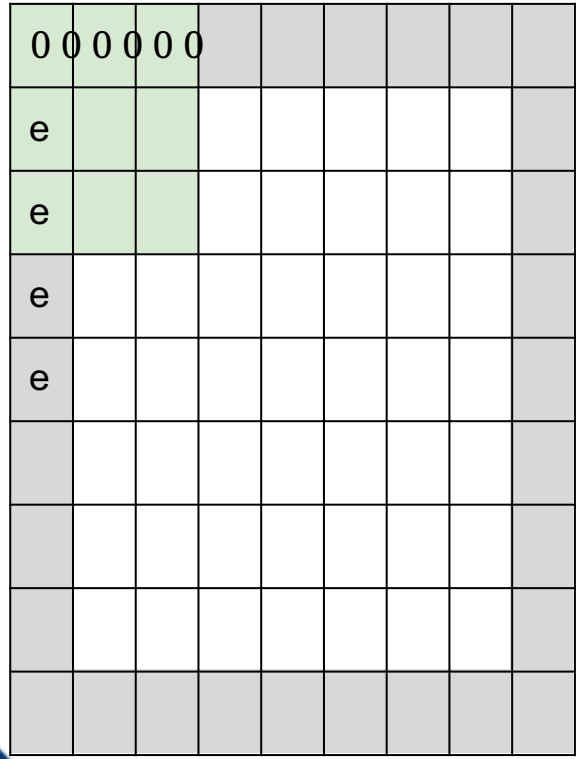
例如，输入 7 7 □□ x□滤波器，应用时使用 **步幅 1，用 1 像素填充** 边界 de 输出是什么?

7x7 输出!

（回顾：） (N ， 3□ -

i) 0stri□e ， 1

实践中：通常对边界进行零填充



例如，输入 7 7 □□ x□滤波器，应用时采用 **步幅 1，填充 1 像素** 边界 de 输出是什么?

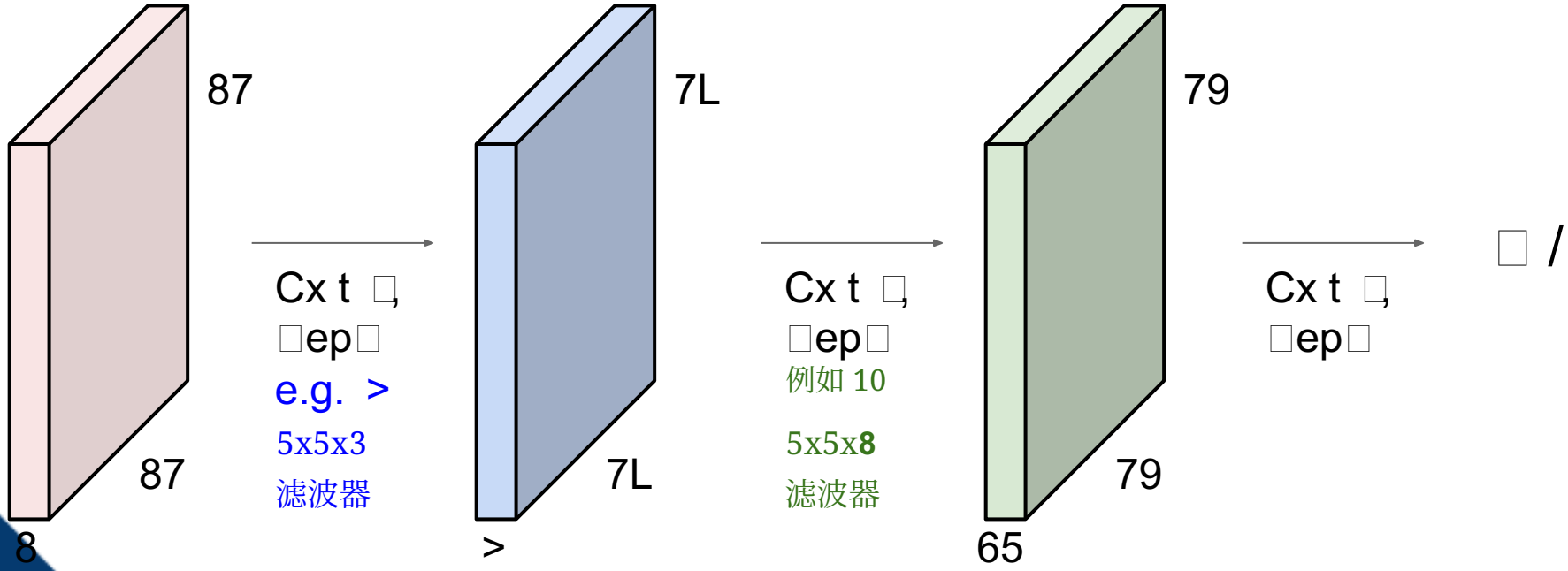
7x7 输出!

通常，常见的是步幅为 1、滤波器大小为 FxF、零填充为 (F-1)/2 的卷积（CONV）层。（将在空间上保持尺寸）

e.g. i d > de zero pad □ith 1
i d F de zero pad □ith 2
i d [de zero pad □ith >

回想一下……

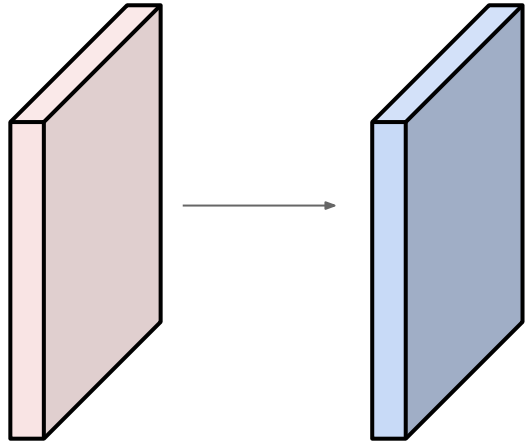
例如，32x32 的输入反复与 5x5 滤波器做卷积会使体积在空间上缩小！（32 -U 7L - U 79 ...）。压缩得太快并不好，效果也不佳。



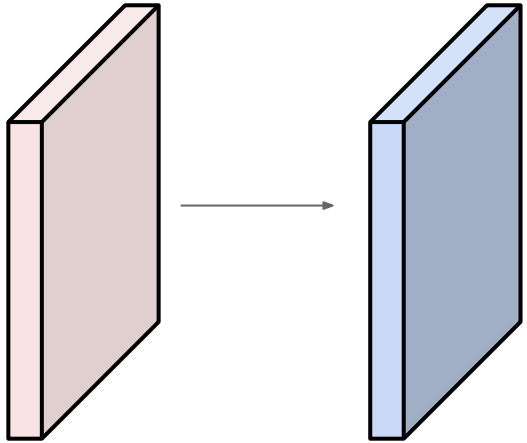
示例时间：

输入体积：**323** 10 5**32**5 **xx**x 个滤波器，步幅为 1，填充为 2

输出体积大小：？



示例时间：



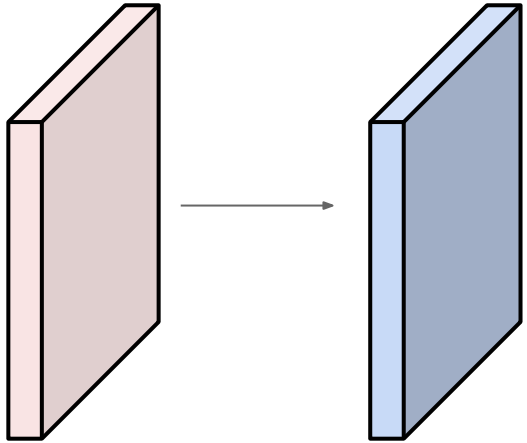
输入体积：323 53 =32= xx□滤波器，步长为 5，填充6

输出体积大小： (76- 6+6-=*25- 5 M76在空间上，因此32x32x10

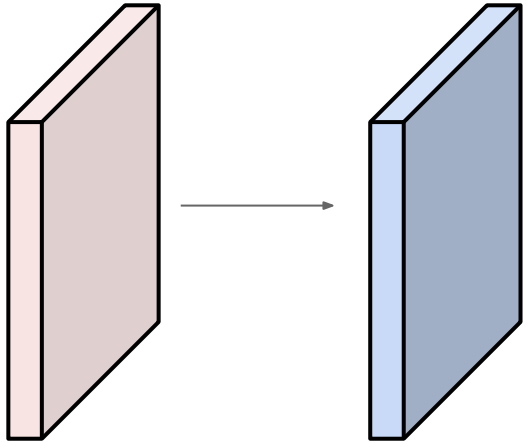
示例时间：

输入体积：**323** 10 5**32**5 **xx**x 个滤波器，步长为 1，填充为 2

这一层有多少个参数?



示例时间：



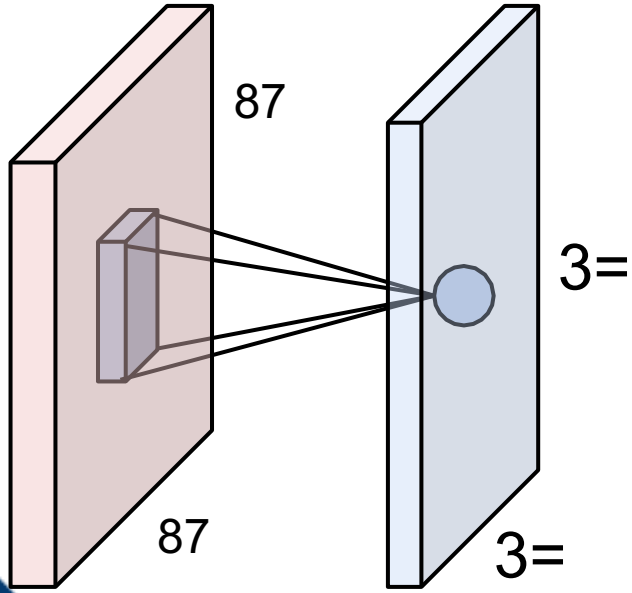
输入体积：32x32x3

53 =□= 步幅为 1、填充为 2 的滤波器

该层的参数数量是多少？

每个滤波器有 =+=+7 - 5 ME> 参数 M> E>+53 M760 (. 1 用于偏置)

感受野



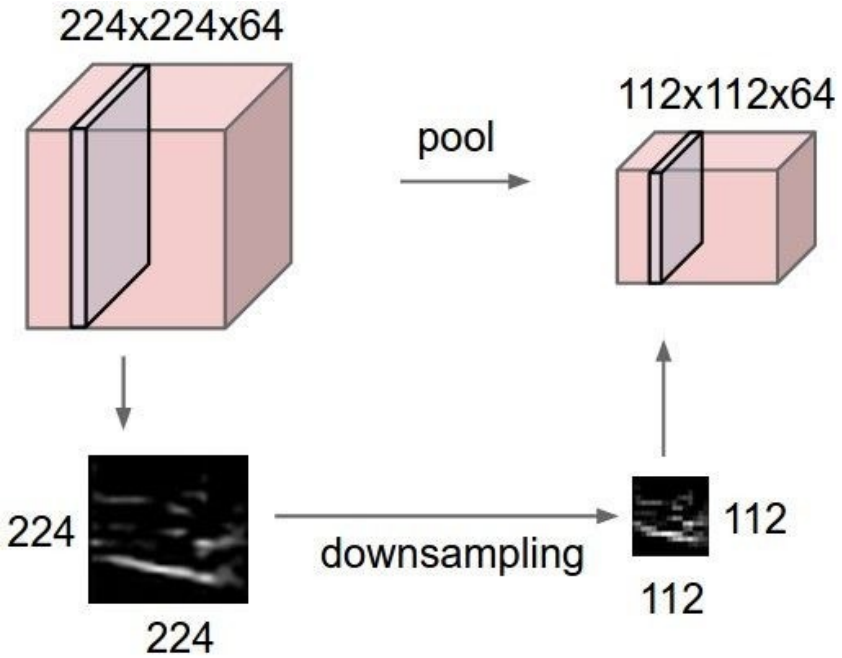
激活图是一张 28x28 的神经元输出表：

1. 每个都连接到输入中的一个小区域 2. 它们全部共享参数

“5x5 滤波器” -U“每个神经元的 5x5 感受野”

https://distill.pub/2019/computing-receptive-fields/

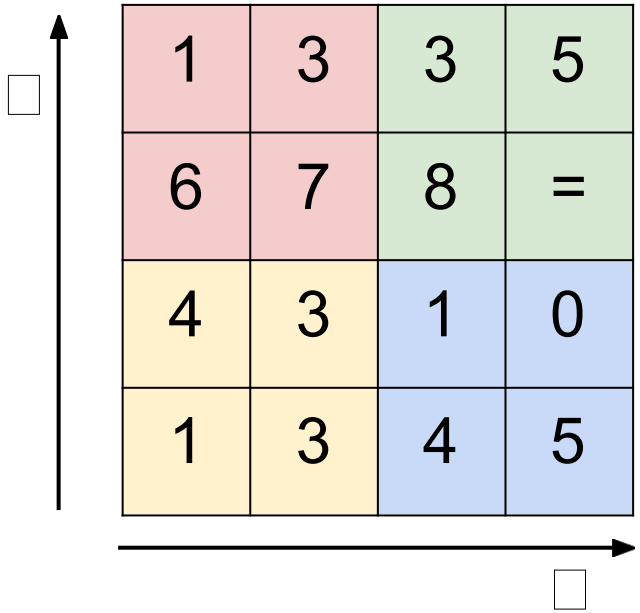
池化层



- 使表示更小且更易于管理 - 对每个激活图独立进行操作：

最大池化

单深度切片



使用 2x2 滤波器和步幅 2 的最大池化

7	=
4	5

匿名课堂反馈

Q&A